neOMAP: High-Performance Oblivious Map for Hardware Enclaves

Anonymous Submission
Anonymous Institution

Abstract

We design and implement neOMAP, a high-performance doubly oblivious map for hardware enclaves. neOMAP combines a deamortized bucketed Cuckoo hash table with an optimized Circuit ORAM, avoiding both periodic rebuilds and the concrete overhead of self-balancing tree. We introduce new parallelization techniques for both batches and dependent sequential requests. neOMAP also uses double tree-top caching and a hybrid external-memory layout to reduce page transfers for maps that exceed physical memory.

For a map of 2^{24} records with 8-byte keys and 56-byte values, neOMAP answers 47K queries/s when each query must wait for the previous answer, 10.6× the rate of Signal's production oblivious map, and breaks even with H_2O_2RAM (USENIX '25), the state-of-the-art hierarchical ORAM implementation. When the queries can be batched, neOMAP further raises throughput to 340K queries/s. At an offered load of 35K queries/s on a map of 2^{27} records, neOMAP responds in 27 μs on average and H_2O_2RAM in 6.0 s, as the latter requires periodic rebuild. We provide C++ and Rust implementations, application demos, and a benchmarking framework.

1 Introduction

An oblivious map [53, 55] allows a user to query and update a key–value store without revealing which keys are accessed. It belongs to a broader family of privacy-preserving techniques called oblivious algorithms, introduced by Goldreich and Ostrovsky [30, 31], that hide a computation's data-access patterns. Over the years, oblivious algorithms have moved from theory into large-scale, real-world systems. For example, Signal's private contact discovery service uses an oblivious map built with hardware enclaves [17, 37].

In an enclave-based deployment, the oblivious map runs inside a trusted execution environment (TEE), such as an Intel SGX enclave [37]. We call the memory protected by the enclave *trusted memory*, which holds the code, the secret keys, and as much of the map as fits. A map that outgrows the machine's physical memory keeps the rest as encrypted pages in *external memory*, an SSD in our deployment, and the enclave fetches a page whenever it needs a record stored on it. Encryption protects the contents of the records, but not the pattern in which they are touched. The server's operating system can observe which encrypted pages move between external and trusted memory [56], and an attacker on the same machine can infer which locations are accessed within trusted memory through processor-cache side channels [13, 21, 23, 34]. The map must therefore hide both the page-level trace between trusted and external memory and the finer-grained trace within trusted memory.

Existing practical oblivious maps [17, 19, 44, 53, 57] leave three performance gaps. First, maps based on self-balancing binary search trees [44, 53] incur high concrete overhead, and the hierarchical-ORAM design of H_2O_2RAM [57] pays high worst-case overhead on periodic rebuilds. Second, existing parallel tree-based constructions [17, 19] use multiple cores only for independent requests in batches, rather than requests that depend on the preceding responses. Finally, most systems [17, 19, 44, 57] do not optimize for maps that outgrow physical memory. Enigmap [53] optimizes for this case, but its AVL tree limits the efficiency of its caching mechanism.

We present neOMAP, a practical oblivious map that addresses all three problems. neOMAP indexes its records with a bucketed Cuckoo hash table over Circuit ORAM, avoiding both periodic rebuilds and the concrete overhead of a self-balancing tree. It parallelizes both dependent sequential requests and batch requests. When the map outgrows physical memory, it also incurs fewer page swaps than prior work. Table 1 summarizes the resulting asymptotic costs.

1.1 Contributions

Circuit ORAM and Cuckoo hashing. neOMAP uses Circuit ORAM, avoiding the periodic rebuilds of hierarchical ORAM while improving asymptotic work and concrete performance over maps built on Path ORAM. Its bucketed Cuckoo hash table provides a further concrete improvement: each

Table 1: Asymptotic comparison of practical oblivious maps. The costs are per map operation. N is the database capacity in records, W is the number of data words in a full record, B is the number of data words per page, and M is the capacity of the in-memory buffer, in data words. A page swap transfers a page between external memory and that buffer. We assume $W < B$. Unless otherwise noted, $\log(\cdot)$ denotes $\log_2(\cdot)$ throughout the paper. $O^*(f(N))$ denotes $O(f(N)g(N))$ for an arbitrarily slowly growing $g(N) = \omega(1)$. The H_2O_2RAM rows describe the released implementation, whose rebuild uses bitonic sorting. "Dependent requests" means that a scheme can use multiple cores even when a request depends on the preceding response. "Batch" means that it can use multiple cores for requests supplied together.

	work	**page swaps**	**parallelism**
neOMAP	$O^*(\log^2 N + W \log N)$	$O(\log_B \frac{N}{M} \log \frac{N}{M} + \log_{B/W} \frac{N}{M})$	dependent requests & batch
Enigmap [53]	$O^*((\log^2 N + W \log N)(\log\log N)^2)$	$O(\log_{B/W} \frac{N}{M} \log N)$	none
Signal [17]	$O^*((\log^2 N + W \log N)(\log\log N)^2)$	$O(\log^2 \frac{N}{M})$	batch
H_2O_2RAM [57]	avg. $O(W \log^3 N)$ worst $O(WN \log^2 N)$	avg. $O(\frac{W}{B} \log^3 \frac{N}{M})$ worst $O(\frac{N \cdot W}{B} \log^2 \frac{N}{M})$	dependent requests

lookup probes two Cuckoo buckets instead of traversing a self-balancing search tree. Insertion is deamortized and evictions follow a fixed schedule, so every operation performs the same amount of maintenance work. Concretely, in a bare-bones ORAM comparison, Circuit ORAM sustains 10.4–15.2× the read throughput of the Path ORAM used by Signal, in a 1.9× smaller memory footprint.

Parallelism for dependent requests and batches. We design parallelization techniques for both dependent requests and batches. Deferred eviction allows neOMAP to return a lookup result while postponing ORAM evictions. For dependent sequential requests, neOMAP probes its two metadata tables concurrently and pipelines reads and evictions across recursive ORAM levels. For batches, neOMAP uses PRF-based sharding and an oblivious compaction-and-expansion router to hide duplicate keys and shard loads. We further develop an improved oblivious router that is 1.44–2.11× faster than Snoopy [19].

Double tree-top caching and external-memory layout. neOMAP caches the small ORAMs at the top of its logical position-map recursion and the top levels of the remaining physical ORAM trees. This double tree-top caching reduces metadata page swaps from $O(\log_B(N/M)\log N)$ to $O(\log_B(N/M)\log(N/M))$. We also develop a hybrid layout tailored to Circuit ORAM that exploits locality in both individual paths and across consecutive evictions, resulting in up to 1.55× less page swaps compared to the layout used in Enigmap [53].

Evaluation. For $N = 2^{24}$ records with 8-byte keys and 56-byte values, neOMAP serves 47K dependent sequential lookups per second, 10.6× Signal's rate and close to H_2O_2RAM's 48K mean. H_2O_2RAM reaches that mean only by amortizing periodic rebuilds that stall every request arriving while one runs. At an offered load of 35K queries/s on a map of 2^{27} records, H_2O_2RAM responds in 6.0 s on average and neOMAP in 0.027 ms, a factor of 2.3×10^5. When requests are independent, neOMAP's batched configuration raises throughput to 340K queries/s, a 20× improvement over Signal's batched implementation and a 7.1× improvement over H_2O_2RAM. With 10^9 records and a 64 GB in-memory page buffer, neOMAP's batched configuration sustains 31K queries/s on one machine.

Implementation and open source. We provide implementations of neOMAP in both C++ and Rust. For reproducibility, both are available as open source at sanitized URLs for the anonymous review process:

- neOMAP C++ version, available at https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/neomap-cpp
- neOMAP Rust version, available at https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/neomaprust

Applications and benchmarking framework. Our open-source artifacts also include two real world applications: an ERC20 token-balance checker and a private server for Ethereum `getProof` requests. Both of them use neOMAP as a backend for their operations. Additionally, we provide a unified framework for benchmarking existing open-sourced ORAM and Oblivious Map implementations. (Section B.2)

1.2 System Model and Security Goal

System and workload. We consider a server that hosts an encrypted map, running neOMAP inside its TEE. Clients send

fixed-length requests directly to the enclave over authenticated, encrypted channels. The map supports lookup, insertion or update, and erasure. Keys and values are padded to fixed, public lengths. We support both *i)* sequential requests, which may depend on preceding responses, and *ii)* public-sized batches of independent requests. Section 3 explains the batch semantics.

Adversary and trust assumptions. We consider a malicious server that controls the OS, thread scheduling, network delivery, and all memory outside the enclave. It may observe, delay, or drop messages and tamper with external pages. Clients may choose keys adversarially and collude with the server. We trust the TEE hardware, its attestation mechanism, the enclave code, and the cryptographic primitives. Attestation lets clients verify the enclave code. A client's authenticated channel prevents the server from modifying or replaying a request.

Observed traces and allowed leakage. The server observes two memory traces: the page-level trace between trusted and external memory, and the finer-grained address trace generated inside trusted memory. The latter models cache-timing attacks that expose accesses within the processor cache [21, 23] and includes the traces of executed instructions. Both traces must hide the requested keys and values, as well as whether each key is present. For a batch, they must additionally hide which requests carry the same key and how many distinct keys the batch contains. We do allow the server to learn the public system parameters, the operation type, the batch size, the fixed response length, all timing information, and whether any of our provisioned internal capacities was exceeded. Our initialization additionally requires the input keys to be distinct, and under that assumption its trace hides the keys, the values, and their order (Section 4.2).

Security goal. For any two valid workloads with the same allowed leakage, the joint distributions of their page-level and within-trusted-memory traces are computationally indistinguishable. Our data obliviousness definition follows Chan et al. [15]. Requiring this property for both traces yields the *double obliviousness* notion of prior enclave maps [39, 44, 53].

We tune neOMAP so that, for each logical request, the probability that any component it touches overflows is at most 2^{-64}. We do not consider vulnerabilities in the TEE itself or leakage from application code outside the map.

Performance metrics. We count both *computational work*, the CPU time consumed by a request, and *page swaps*, the encrypted pages transferred between external memory and the in-memory buffer (Section 4.1). We report both because an implementation can trade one for the other, saving CPU work while moving more pages, or the reverse. We also measure end-to-end latency, throughput, and storage overhead.

2 Our Oblivious Map Algorithm

This section presents neOMAP's core algorithms. Section 2.1 recalls Circuit ORAM [54] and describes where our instantiation departs from it. Section 2.2 then combines it with deamortized Cuckoo hashing [7, 45] to build the oblivious map.

2.1 Circuit ORAM and Our Instantiation

An Oblivious RAM (ORAM) [30, 31] implements an array interface such that the physical addresses an access touches are independent from the logical indices requested. While an ORAM indexes a dense address space rather than the sparse key space of a map, we will use it as a building block for neOMAP.

ORAM tree and path invariant. Circuit ORAM follows the tree-based framework introduced by Shi et al. [50]. A single tree, also called a *non-recursive ORAM*, stores blocks identified by logical addresses. The tree has enough leaves for its public logical capacity. Each node is a *bucket* containing a constant number of slots, and each slot contains either a real block or a dummy block denoted $\perp$. The root bucket is larger and serves as a stash.

To hide a block's location, Circuit ORAM assigns each real block a uniformly random leaf, or equivalently the root-to-leaf path ending at that leaf. This assignment is the block's *position identifier*. The *path invariant* requires the block to reside somewhere on its assigned path. Because the root lies on every path, a fetched block can always be reinserted there before later evictions move it downward.

Circuit ORAM access. With this invariant in place, an access has three steps. First, the caller supplies the requested block's logical address addr and current position identifier pos. The operation $\mathsf{ReadRm}(\mathsf{addr}, \mathsf{pos})$ scans the entire path, obliviously selects the requested block, and removes it. Second, the caller reads or updates the block, assigns it a fresh random position identifier, and reinserts it at the root. The fresh assignment ensures that a later access to the same block only reveals an independently random path.

Finally, Circuit ORAM evicts blocks from the root toward the leaves to prevent the root from overflowing. On an eviction path, its circuit first determines which blocks can move downward and then scans the path while holding at most one block at a time. Every move respects the block's assigned path, so eviction preserves the path invariant. After each access, Circuit ORAM performs two such evictions on paths selected in a public, deterministic reverse-lexicographical order [54].

Why Circuit ORAM rather than Path ORAM? The main distinction between Circuit ORAM and Path ORAM [52] is the eviction algorithm. Path ORAM, used by several earlier systems [1, 17, 44, 53], greedily packs eligible blocks as close to the leaf as possible. This approach is attractive in the traditional client–server setting, where communication dominates and the client can rearrange a downloaded path locally.

The tradeoff is different inside a TEE. Memory is local, so the computation performed during eviction matters more. The CPU cache cannot serve as a large private buffer, because under our double obliviousness goal the rearrangement itself must hide its memory trace. Common TEE implementations rearrange a path with L slots using a bitonic sorter, which incurs $O(L\log^2 L)$ work [1, 10, 17, 44, 53]. Circuit ORAM instead uses a constant number of linear scans and a one-block working buffer, reducing the eviction work to $O(L)$. Concretely, replacing Path ORAM with Circuit ORAM alone accounts for roughly a $5\times$ latency improvement, as our breakdown benchmark shows (Section C.2).

Recursion. The description above assumes that the caller already knows the requested block's position identifier. A recursive ORAM removes this assumption by storing position identifiers in progressively smaller ORAMs until the remaining position map is small enough to scan obliviously inside the enclave [50, 52]. If one position-map record contains $\chi = O(1)$ identifiers, each occupying one data word, the recursion has $O(\log_\chi N)$ levels. The number and sizes of these levels depend only on the public capacity and system configuration.

neOMAP's eviction schedule. Circuit ORAM leaves the choice of eviction paths to the implementation, as long as that choice is independent of the logical addresses. neOMAP uses this freedom to extract more eviction work out of each path it fetches. For sequential requests, after reinserting the block with a fresh position identifier, neOMAP makes a full eviction pass over the path just read. That path is still buffered, so the pass costs computation but no extra path fetch. neOMAP then takes the path at the current position of its public reverse-lexicographic counter, makes *two* consecutive eviction passes over it and writes it back. An access therefore performs three eviction passes while touching only two paths. The second deterministic pass operates on the path state left by the first, and both deterministic passes follow the reverse-lexicographical order used by Circuit ORAM [54] and earlier work [27, 28].

With the third eviction pass, the tail of the measured stash-occupancy distribution decays substantially faster than under the standard two-eviction schedule. Section C.1 reports these tails and the resulting stash capacities.

Our scheme for batched requests uses a slightly different eviction schedule, which we describe in Section 3.4.

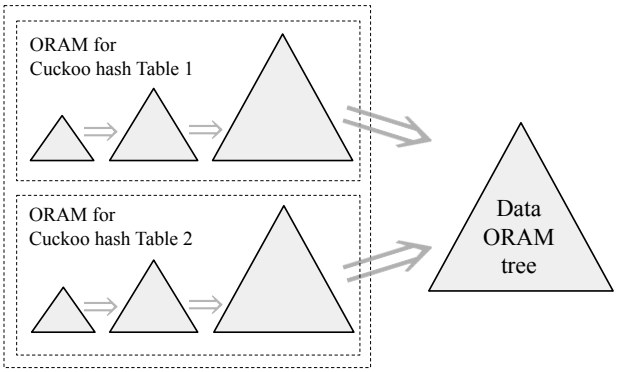


Figure 1: neOMAP stores full records in one non-recursive data ORAM. Two recursive Circuit ORAMs hold the two Cuckoo metadata tables, whose entries map keys to data-ORAM paths. Every operation accesses both candidate metadata buckets and one real-or-dummy data path.

2.2 Our Oblivious Map

neOMAP's oblivious map interface has three operations:

- $\mathsf{Lookup}(k)$, which returns the value for k or $\perp$;
- $\mathsf{Insert}(k, v)$, which replaces the value if k is present and otherwise creates a new entry; and
- $\mathsf{Erase}(k)$, which removes k if it is present.

The value and membership result are returned over the client's encrypted channel in a fixed-length response. Operation type is public, but membership is not. Keys and values are padded to fixed lengths.

Oblivious lookup and erasure with Cuckoo hashing. Cuckoo hashing [45] is well suited to an oblivious map. While a hash table with linear or quadratic probing walks a probe sequence on lookup to find the key, the number of probes depends on the key and on which other keys collide with it — an oblivious implementation would therefore have to pad every lookup to the longest probe sequence in the table. By contrast, Cuckoo hashing leaves no such sequence to pad. We use two candidate buckets per key, one from each hash function, and each bucket holds two entries. A lookup always probes both candidate buckets, no matter which one holds the key — or whether the key is present at all.

Erasure fits the same pattern: it probes both candidate buckets and the stash exactly as a lookup does, then clears the matching entry with a branchless conditional mov.

Cuckoo insertion cannot always place an entry in the table, so the table carries a small stash for this purpose [38, 43], and every lookup and erasure linearly scans that stash in full.

Keeping the memory overhead low. Keys and values can be much larger than the position information needed to locate a record. Storing complete records in the Cuckoo tables would make every bucket larger, amplify the storage consumed by empty Cuckoo slots, and make every probe and displacement move a full record. We therefore keep the records out of the tables. As Figure 1 shows, neOMAP stores each complete (k, v) record in a dense, non-recursive data ORAM. To find a record, it keeps compact entries that map keys to data-ORAM position identifiers in two separate tables. Each table is implemented as a recursive Circuit ORAM. The two candidate locations for a key are one bucket in each table.

The standard Cuckoo construction stores one entry per bucket, whereas we use the bucketized variant and set the bucket size to 2. Each recursive-ORAM bucket incurs the position-map cost needed to locate it, so storing more Cuckoo entries in a bucket amortizes that cost over more keys. Two entries per bucket also let us fill the table much further: with single-entry buckets, two-choice Cuckoo hashing saturates at a load factor of about 0.5, whereas buckets of two reach roughly 0.9 [22]. We choose 0.7, which balances memory utilization against insertion cost.

Deamortized Cuckoo insertion. Insertion, however, is more complicated for oblivious cuckoo hashing. Standard Cuckoo hashing places a new entry into one candidate bucket and, if that bucket is full, repeatedly displaces resident entries to their alternate buckets. The resulting replacement chain can be long, and its length reveals membership: an update to a key already in the map could stop as soon as that key is found, whereas inserting a new key may start such a chain. A direct oblivious implementation would therefore have to pad every insertion to the maximum chain length.

To avoid paying this cost, we spread the replacement work across operations using the FIFO-queue deamortization paradigm of Arbitman et al. [7]. The pending queue has a public, fixed capacity chosen from the empirical tail analysis in Section A.1, and every map operation scans it in full. The queue also replaces the stash: an entry that cannot be placed stays queued, so an operation scans one structure rather than two. A new entry is appended to its tail. Each insertion then performs the same public constant number $R = O(1)$ of relocation iterations, including when it merely updates an existing key. In each iteration, the algorithm removes the current head and tries to place it in Table 0, appending any displaced entry to the queue tail. It then removes the new head and repeats the same step for Table 1. An entry evicted from Table 0 therefore returns to the queue before the Table 1 step selects its entry. A step with no real entry performs the corresponding dummy work. Figure 2 illustrates this fixed-work flow.

An insertion is logically complete after these R rounds even if its metadata has not yet reached a Cuckoo bucket. The entry remains live in the pending queue while later insertion rounds continue placing queued entries. When a full bucket requires an eviction, either of its two entries can be displaced, and we choose the one whose alternate bucket appears less congested, so that it is more likely to find space after displacement. This helps keep the pending queue small without increasing the number of relocation rounds. Section A.1 specifies the policy and reports the resulting queue-occupancy tail.

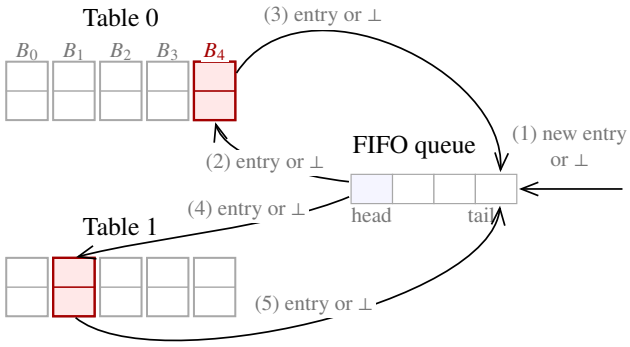


Figure 2: Deamortized cuckoo insertion [7]. The new entry is appended at the FIFO tail. Each fixed-work iteration processes one queue head in Table 0 and then the next queue head in Table 1, appending any evictions to the tail. Each table is implemented as a recursive Circuit ORAM as in Section 2.1.

Cost and security. Using W for the number of data words in a full record, as in Table 1, each position-map record holds $O(1)$ positions, each occupying one data word. The position maps therefore contain only compact metadata, and their recursive accesses contribute $O^*(\log^2 N)$ work, whereas the data-ORAM access contributes $O(W \log N)$. The two metadata probes and one data probe together give $O^*(\log^2 N + W \log N)$ work and linear storage. The deamortized Cuckoo hashing analysis gives a pending queue capacity of $Q = O(\log N)$ for a negligible failure probability [7], so this scan adds $O(\log N)$ work and does not change the bound above.

Conditioned to neither ORAM-root nor pending-queue overflow, logical map semantics follow from the ORAM path invariant – empirical failure analyses and concrete capacities appear in Sections A.1 and C.1. The queue scan, two metadata accesses, data access, pointer refresh, and relocation rounds have public lengths. Candidate addresses and real data paths are PRF outputs hidden by their ORAMs, while dummy paths have the same uniform distribution. Thus these steps reveal no more than the leakage in Section 1.2.

3 Parallel Request Processing

neOMAP uses different forms of parallelism for the two workloads introduced in Section 1.2. When requests arrive one at a time, it overlaps work within each recursive ORAM access.

When requests arrive as a batch, it partitions the map into shards that run in parallel.

3.1 Sequential and Batched Requests

We first define the two workload interfaces.

- *Sequential workload.* The caller submits one request at a time and issues the next only after the current one has returned. This is the standard RAM model, and it is what a generic program needs to run obliviously on the server. Because the next request may depend on any earlier response, neOMAP cannot move work across requests speculatively.
- *Batched workload.* The caller submits a group of requests at once, with only one group active at a time. A batch is a sequence $q_1, \ldots, q_m$ of requests of a single operation type. The batch size m and the operation type are public, whereas the keys, the values, and the pattern of repeated keys are not. The result is defined relative to the state at the start of the batch: a lookup returns the value bound to its key at that point, so two requests carrying the same key receive the same value; among the insertions carrying a given key, only the last one takes effect; and an erasure removes every key that occurs in the batch.

Responses have the fixed length described in Section 2.2. An insertion therefore returns the same kind of acknowledgment whether the key was new, was already present, or was inserted again later in the same batch. Once the enclave has saved every result and update, it can return the responses, but the ORAM must still finish its required write-backs and evictions, work we call *maintenance*. Responses may be used while maintenance runs, but the service waits for it to finish before starting the next batch.

Restricting a batch to a single operation type does not limit the applications we target, since they already separate updates from queries in time. Signal's contact discovery service rebuilds its database periodically rather than on every user request [17], and our Ethereum state-server application likewise separates state updates from read-only queries.

3.2 Sequential Workload: Pipelining Across ORAM Levels

Recall from Section 2.2 that every map operation accesses the two recursive metadata ORAMs and then the data ORAM, and that each metadata access traverses several ORAM levels. An insertion also performs a fixed number of relocation rounds, one after another. The optimization below overlaps work within an individual recursive access, and it leaves the order of an insertion's steps unchanged.

Independent metadata tables. The two Cuckoo tables reside in separate recursive ORAMs, so their bucket probes share no ORAM state and run concurrently.

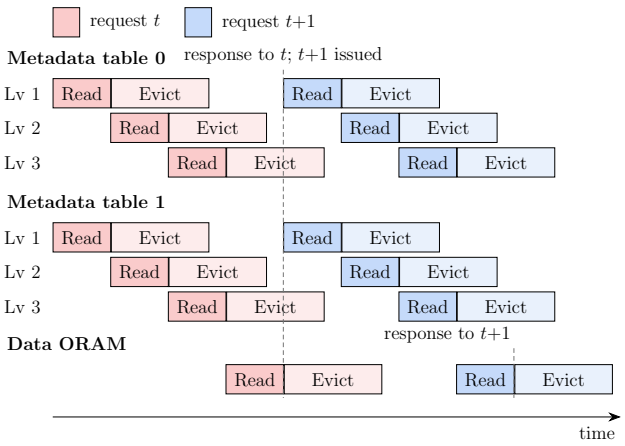


Figure 3: Pipelined execution of two consecutive sequential lookups. Each row is one recursion level, owned by one thread, and the two metadata tables are probed concurrently. A level hands its result to the next level after its read pass and then evicts. The response to request t leaves as soon as its data-ORAM read complete, and its eviction may overlap with the reads of request $t+1$.

Pipelining recursion levels. The second form of overlap is possible because a level finds the information needed by the next level before it begins eviction. At recursion level ℓ, the system reads the path containing the position-map node and obtains the child's current position identifier. It also samples a fresh identifier, writes the updated node at the root of the level-ℓ tree, and then evicts that level. The next level needs only the child's *old* identifier, so it can start as soon as the read at level ℓ finishes, without waiting for that level's eviction.

neOMAP therefore assigns each recursion level of each metadata table, as well as the data ORAM, to its own thread. When a thread finishes the read pass of its level, it hands the identifier to the thread for the next level and then performs its own evictions. Because the recursion depth and the level sizes are public, the number of threads is public as well. Section A.2 gives the concrete count for our configuration.

Figure 3 shows the resulting schedule for lookups. A lookup response is returned as soon as the data-ORAM read completes, which is also the earliest moment at which the caller can issue request $t+1$. The unfinished evictions of request t, including the data-ORAM eviction, then overlap request $t+1$'s execution. Thus evictions stay off the steady-state critical path, provided that each eviction finishes before the next request reaches its level. Each level is still internally serialized, so no ORAM tree is mutated concurrently.

An insertion decomposes into several accesses to the underlying ORAMs, so it benefits from the same pipelining.

Security. The adversarial server may pause or resume threads, delaying when an identifier passes between levels but

changing neither the work nor the memory locations used. At any given level, all requests perform the same public schedule and pass one identifier to the same next level. Scheduling thus affects only timing. Addresses stay independent of secret keys and values, and the ORAM accesses remain oblivious.

3.3 Batched Workload: Oblivious Routing to Shards

A batch contains m independent requests, so neOMAP can process it faster by partitioning the map into n independent *shards*. Each shard is an instance of the design in Section 2.2, and a separate worker processes the requests for that shard. The n workers can then run in parallel.

The routing to these workers must itself be oblivious. If the server observed which shard receives each request, or how many requests each shard receives, it could learn correlations among the requested keys. Every worker must therefore receive the same public number of requests, regardless of how many real requests it must process.

Snoopy [19], a prior oblivious storage system, solves this problem using oblivious sorting and compaction. It maps each key to one of the n shards by evaluating a secret PRF on the key. Let c be the public number of requests that each shard will process. Because compaction can remove records but cannot create missing positions, Snoopy appends c dummy requests per shard before sorting, or $m_{\text{pad}} = nc$ dummies in total. After sorting, Snoopy marks one request for each distinct key and enough dummies to select exactly c records per shard, provided that no shard has more than c distinct keys. It then compacts away the rest.

Our routing optimization. Snoopy's padding makes its two oblivious sorts operate on $m + m_{\text{pad}}$ records. Our enclave-resident router uses oblivious expansion instead, so both sorts operate on only the m original requests.

The first sort orders the requests by shard, key, and original batch position, which groups each shard's requests and places repeated keys next to one another. A linear scan then marks one request for each distinct key, while the other records remain in the router until their answers can be filled.

The router gathers the marked requests into a prefix with stable compaction (OrCompact [49]). Stable expansion (OrExpand [46, 49]) then creates n fixed-capacity ranges of c requests and fills unused positions with dummies. Every worker receives exactly c requests, and dummy requests follow independent uniformly random paths (Section 3.4).

Compaction and expansion are reversible. After the workers finish, the router runs both networks backward, copies each representative's answer to the other requests for the same key, and performs its second sort to restore the original batch order. The resulting router is 1.44–2.11× faster than the Snoopy baseline across the evaluated batch sizes (Table 3).

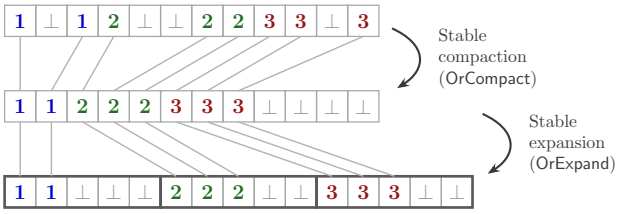


Figure 4: Routing sorted requests to shards, here $n = 3$ shards of capacity $c = 5$. The numbers denote shard assignments, and $\perp$ denotes the inactive routing payload of a duplicate or a dummy request. Oblivious stable compaction (OrCompact) gathers the live representatives, and stable expansion (OrExpand) places them into the fixed-capacity shard ranges outlined in bold. Section A.2 describes both routing networks.

Security and full algorithm. Conditioned on no capacity failure, the fixed networks and equal-size shard inputs reveal nothing about request destinations, the duplicate pattern, or the number of real requests sent to each shard. We choose the public capacity c so that the probability that any shard receives more than c distinct keys stays within the router's share of the 2^{-64} per-request failure budget of Section 1.2. If this limit is exceeded, the system reports the failure and aborts the batch. Section A.2 gives our full batching algorithm as well as the derivation of c.

3.4 Executing a Batch Inside a Shard

Per-request baseline. After routing, the simplest approach is for each shard to run the ordinary recursive ORAM algorithm on its c requests, one at a time. Each access would read the recursive paths, reinsert the updated nodes, and finish the scheduled evictions before starting the next request. Our sequential schedule hides much of this maintenance latency by assigning a thread to every recursion level. In a batched execution, however, the cores are divided among the shards. A shard can probe its two metadata tables concurrently, but it does not have enough cores to pipeline all their recursion levels. With the simple per-request schedule, write-back and eviction therefore remain on the response critical path.

Two-phase execution. Each shard processes a batch in two phases. In the response phase, the shard handles all c requests, including dummy requests. It reads the required ORAM paths, computes the results inside the enclave, and saves modified nodes there instead of writing them back immediately. Each ORAM tree schedules exactly c write-backs for maintenance. Once all response values are ready, the shard returns them.

The maintenance phase runs during the interval between batches. It completes the scheduled write-backs and performs the required Circuit ORAM evictions. The service starts the

next batch only after this phase finishes. Separating the phases moves write-back and eviction off the response path rather than eliminating this work, and the measurements in Section 5 include both phases.

For each scheduled write-back, maintenance fetches two adjacent paths from the public deterministic schedule, inserts the modified node or a dummy at the root, performs one eviction pass on the first path, and two on the second. This provides the three passes needed to control stash occupancy with two path fetches and preserves locality in our external-memory layout (Section 4.1).

Deduplicating shared-node accesses. Routing removes repeated keys, but distinct keys can still occupy the same Cuckoo bucket or visit the same node in a recursive position map. Because its write-back is deferred until maintenance, reading the node separately for every request would reuse the node's old physical path and reveal that the requests collide. The shard instead performs one real access for a representative, while all other requests access independent uniformly random paths and discard the results. Oblivious scans propagate the fetched node within the group and combine all changes into the single copy later written back.

One sort serves all recursion levels. Without further structure, the shard would deduplicate one level at a time, sorting the c requests by their current node at each of the L recursion levels. We avoid these repeated sorts by observing that every recursive node identifier is a prefix of the table's complete bucket index. One oblivious sort by the complete index therefore makes requests for the same node contiguous at every level. Each level then needs only oblivious scans to propagate results and accumulate updates. Each metadata table therefore performs two sorts: one bucket-index sort before the levels and one final sort to restore the original request order. This eliminates $L-1$ sorts per table. The two tables run concurrently and their results are merged with a linear scan.

Security and full algorithm. Conditioned on no capacity failure, every shard performs c path accesses at each metadata level and at the data ORAM. Representatives use their previously sampled random position identifiers, and other requests use independent uniformly random paths. Every tree performs exactly c scheduled write-backs under the public eviction schedule.

Appendix A.2 gives the shard-execution procedure and its security argument.

4 External Memory and Other Optimizations

4.1 External-Memory Design

Memory hierarchy. neOMAP explicitly manages two memory tiers. The in-memory tier holds M data words with word-granular access. Once the map no longer fits there, the remaining state lives on SSD as encrypted, authenticated pages, and reading even a single word transfers a whole page of B data words into a bounded in-memory buffer.[1] This section reduces that page traffic by making each fetched page do more useful work, at no extra computational cost.

Caching both physical and logical tree tops. Following Phantom's tree-top caching and Enigmap's external-memory adaptation [41, 53], we cache tree tops and pack nodes from several levels into a page. The recursive position map of neOMAP offers an additional opportunity. Its logical levels are fixed, and each is stored in a separate non-recursive ORAM, so the small ORAMs near the top of the recursion can stay in memory. By contrast, rotations in Enigmap's AVL tree prevent a record from remaining at a fixed logical level, so its nodes share one non-recursive ORAM.

For a public map configuration, we reserve a fixed margin for request and maintenance buffers and choose the largest cache depth h for which the top h levels of all remaining trees fit in the available memory. Trees with fewer than h levels are cached in full. Caching both the physical ORAM trees and the logical position-map recursion reduces the metadata I/O of a map operation from $O(\log_B(N/M)\log N)$ to $O(\log_B(N/M)\log(N/M))$ page swaps.

Balancing read and eviction locality. Classical subtree packing [5, 53] gives good locality for an individual root-to-leaf path. Circuit ORAM, however, processes a randomly chosen read path and also evicts along paths drawn from a public reverse-lexicographical order: one such path per sequential request, and a window of two adjacent ones per batched write-back (Section 2.1). Laying out each path in isolation forfeits locality across consecutive deterministic evictions. RORAM [14] exploits that deterministic order, but its eviction-oriented layout makes random read paths more expensive.

Our Circuit-ORAM-specific hybrid combines these two ideas. It collapses each depth-d subtree into a contiguous super-node, then places super-nodes in reverse-lexicographical order so that those used by nearby evictions are also nearby in memory. A physical page may contain several super-nodes, since the layout does not require each

[1] Because the backing store is an SSD, a page is simply the hardware page of 4 KB, and $B = 4\,\text{KB}/8\,\text{B} = 512$. Older enclaves such as SGX v1 have far less trusted memory, and may swap data to untrusted DRAM instead. DRAM imposes no page granularity of its own, but transferring at a fixed page size remains beneficial there as well, as discussed in Section A.3.1.

super-node to fill a page. A larger d means a read visits fewer super-nodes, while an eviction fetches more nodes it does not use, so we pick d to balance the cost of the random read path against that of the deterministic eviction paths.

Overall page-swap cost. Under the notation and $W < B$ assumption of Table 1, a position-map record occupies $O(1)$ words, whereas a record in the data ORAM occupies W words. Combining the tree-top caching and sub-tree packing techniques, we obtain a per-request page-swap count of

$$O\Big(\log_B(N/M)\log(N/M)+\log_{B/W}(NW/M)\Big).$$

The first term accounts for the recursive position maps, and the second for the NW words in the data ORAM. Under $W < B$, the expression simplifies to the bound displayed in Table 1. Sharding does not change this asymptotic cost: for n shards and batch size $m = \omega(n \log N)$, each shard has $O(N/n)$ records, $O(m/n)$ requests, and $O(M/n)$ words of in-memory buffer, so n cancels from the per-request page-swap bound.

Concretely, under neOMAP's eviction schedules, the hybrid layout cuts the page count by 1.18–1.55× relative to classical subtree packing (Table 2). Full layout details appear in Section A.3.

4.2 Fast Initialization

Naive initialization inserts every record via $\mathsf{Insert}(k,v)$ — Cuckoo insertion invokes recursive ORAM accesses that repeatedly overwrite the same position-map nodes. However, given that during initialization all the insertions are known in advance, we can avoid this repeated work by building the map in two phases. The first phase determines the final Cuckoo placement, and the second builds the recursive ORAMs holding the completed tables.

Phase 1: Cuckoo placement. The initializer temporarily replaces the recursive ORAM underlying each Cuckoo table with a non-oblivious array. It performs all Cuckoo replacements on these arrays and leaves any entry it cannot place in the pending queue. A replacement therefore costs an array access rather than a recursive ORAM access, so Cuckoo overhead does not multiply with recursive-ORAM overhead. This phase fails only if the pending queue overflows.

The exposed array accesses reveal bucket indices but not the corresponding keys. For distinct input keys, the resulting candidate indices are computationally indistinguishable from independent uniform locations by the security of the PRF. A simulator given only N and the public map parameters can therefore generate an indistinguishable placement trace, including queue behavior. If a key appears twice, however, its candidate indices repeat and reveal the equality, so we require the input keys to be distinct.

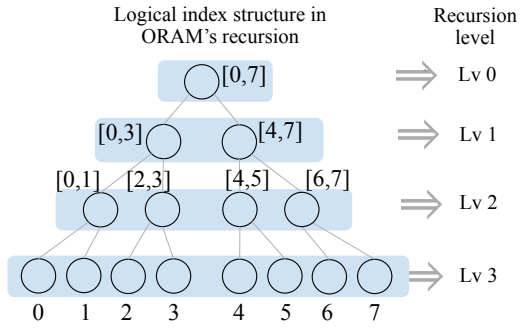


Figure 5: Post-order ORAM initialization. The tree is the logical recursion structure of the ORAM. Nodes at one depth belong to the same recursion level. Building children before their parent makes every child position identifier available when the parent block is formed.

Phase 2: Recursive ORAM construction. The completed arrays specify the final contents of the two Cuckoo tables. A straightforward builder stores these contents in recursive ORAMs one level at a time, starting with the largest ORAM at the leaf level. Placing the blocks at one level produces the position identifiers required by the next level, so the builder must retain all of those identifiers until the current level is complete. The first intermediate array contains $\Theta(N)$ identifiers and therefore requires linear additional workspace.

Our initializer instead constructs all recursion levels concurrently. As soon as all the children of a position-map node have been placed, it combines their position identifiers into the completed node, inserts that node at the next level, and passes the new position identifier upward. Each position-map node is therefore written once, and the algorithm uses only $O(\log N)$ additional node storage. This schedule is effectively a post-order traversal of the recursion structure, as we illustrate in Figure 5.

Cost. Our initialization cost is dominated by building the data ORAM. Under the $W < B$ packing assumption of Table 1, it performs $O^*(NW \log N)$ work and $O(N \log_{B/W}(N/M))$ page swaps in total. In contrast, initialization via naive repeated insertion would require $O^*(N(W \log N + \log^2 N))$ work and $O(N \log_{B/W}(N/M) + N \log_B \frac{N}{M} \log \frac{N}{M})$ page swaps.

In comparison with the oblivious-sorting-based initializer of Enigmap [53], our approach is concretely 4.2× faster at $N = 2^{26}$, and the gap grows with N (see fig. 10 in section 5.4).

4.3 Integrity Verification

An untrusted operating system can attempt to tamper with encrypted pages stored in external memory. AES-GCM authenticates a ciphertext under a given nonce, but it does not by itself guarantee freshness, since the adversary may replay an older valid ciphertext.

Authenticating the ORAM tree. Following prior work [26, 42, 53], we maintain the freshness nonces in a tree and overlap this nonce tree with the ORAM tree, so that only the root nonce remains inside the enclave. In our instantiation, each super-node (as in Figure 13) stores an authenticated counter for each child, recording how often that child has been visited. Since ORAM accesses traverse tree paths, the counter needed to verify a fetched super-node is obtained from its parent. We further save one counter per super-node using the invariant that the child counters sum to the parent counter, because every path that reaches a super-node continues into exactly one child: the parent stores counters for all but one child, and the missing counter is derived on the fly. Compared with Enigmap [53], where each node stores a random nonce along with the nonces of all its children, this further reduces space usage. Relative to Freecursive ORAM [26], we reduce computational work at slightly higher space usage.

Authenticating Initialization. The initializer also keeps its transient pages in the untrusted 4 KB page store. These pages do not yet form the ORAM tree, so the nonce tree does not apply to them. We instead encrypt each transient page with AES-GCM and keep its version number inside the enclave for as long as the page is live, deriving the nonce from that version number together with the build instance and the page index. Tampering with a page, swapping in a different one, or replaying an older version therefore fails authentication and aborts initialization. Holding the version numbers in the enclave costs little trusted memory: one 8-byte counter per 4 KB page, about 0.2% of the page store it protects.

4.4 System-Level Optimizations

Constant-time vector operations. Our ORAM entries are large enough to benefit from vector instructions. We use AVX2 and implement oblivious swaps with 256-bit vector blends (`VPBLENDMQ`).[2] The compiled primitive has fixed-address loads and stores and contains neither secret-dependent branches nor secret-dependent memory accesses.

Custom page manager. While SGX v2 can page enclave memory automatically [24], our external-memory experiments use the application-managed 4 KB page store of Section 4.1. Managing the transfers ourselves lets us pin the physical and logical tree tops in memory, rather than leaving residency to SGX's replacement policy. The page store encrypts and authenticates every page with AES-GCM, and the freshness mechanism of Section 4.3 prevents replay, so it provides the same confidentiality, integrity, and anti-replay properties that we would otherwise need from SGX paging during an enclave execution. It is also faster: on the Ice Lake SGX server used in our evaluation, moving a page between the enclave and main memory is 15.9× faster in our page store than under SGX paging, in a sequential-access microbenchmark that favors SGX's replacement policy.

[2] While our Ice Lake processor supports AVX512, evaluation shows that it takes longer to execute one 512-bit blend than two 256-bit blends.

Secure randomness. We use SGX's default PRNG (`sgx_read_rand`) to sample only a seed and expand it with a custom thread-safe AES-CTR PRNG. This avoids repeatedly executing the expensive `RDRAND` instruction.

5 Evaluation Results

5.1 Experimental Methodology

Machine and enclave configuration. All experiments run inside an enclave on a 32-core Intel Xeon Platinum 8352S machine with 1 TB of DDR4-3200 RAM and a 1.7 TB Intel NVMe SSD, under Intel SGX v2. Each experiment states the Enclave Page Cache (EPC) capacity we give the enclave. Map pages beyond that capacity reside on the SSD. We implemented neOMAP in C++ and Rust, and the measurements below use the C++ implementation. Both are configured with overflow probability at most 2^{-64} for each logical request (summed across all components), which matches H_2O_2RAM [57].

Baselines and configurations. We compare neOMAP with three end-to-end oblivious maps and one bare-ORAM implementation. neOMAP runs in the two configurations of Section 3.1: neOMAP-seq serves one dependent request at a time, and neOMAP-batch-m serves m independent requests together. Every batched neOMAP configuration uses 15 shards, and every batched Signal configuration uses 16.

- **Signal's implementation** [3], written in C and jasmin [6] currently used in production, places a robin-hood hash table on top of Path ORAM [52] and builds it from formally verified oblivious primitives. It provides one-sided security: lookups are oblivious, but insertions are not as the server supplies the user database itself. neOMAP protects both. We benchmark the production code as shipped.
- **H_2O_2RAM** [57], a C++ implementation of hierarchical ORAM is the baseline with the fastest throughput to our knowledge. However, H_2O_2RAM is amortized, so its worst-case latency is much higher than its average latency.
- **Enigmap** [53], a C++ implementation that places an AVL tree on top of Path ORAM. It is the first oblivious map with an ORAM-tree layout designed for external memory, making it the closest baseline for our page-swap optimizations.
- **Meta's implementation** [1], a Rust implementation of Path ORAM. It does not include a map layer. We use

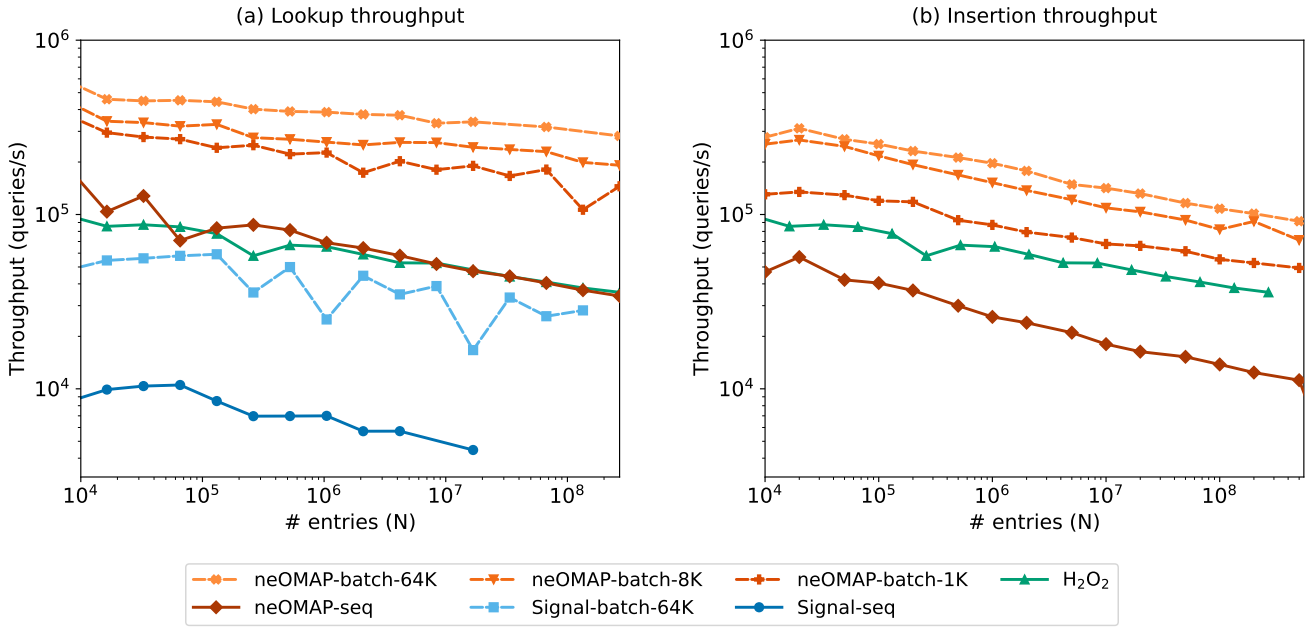


Figure 6: Lookup and insertion throughput (8-byte keys, 56-byte values) for sequential and batched workloads. For batched implementations, the suffixes represent the batch size. For H_2O_2RAM, their insertion time is the same as lookup by the property of hierarchical ORAM. Signal is compared in Section C.3.

it only for the bare-ORAM comparisons in Figures 16 and 17.

Datasets. We vary the number of records N and take the key and value sizes from our target applications. Based on publicly reported user counts [47], we estimate Signal's contact discovery database at roughly 2^{27} records with 8-byte keys and 56-byte payloads.

Metrics. We issue requests to every system through its own API, which takes a single request or, where the implementation supports it, a public-sized batch of independent requests. The operations are lookup, insertion or update, and erasure. We report four metrics.

- *Saturated throughput* is the highest request rate an API sustains over a sufficiently long windows.
- *End-to-end latency* is the time from a client submitting a request until the API returns the response, so it also covers any wait before the map begins serving that request.
- *Memory overhead* is the footprint of a populated map divided by the raw size of the N key–value records it holds.
- *Initialization time* is the time for populating a map from a dataset of N records.

5.2 Saturated Throughput

Sequential Lookups. At $N = 2^{24}$, neOMAP-seq serves 47K lookups per second, 10.6× Signal's 4.5K (Figure 6(a)). At $N = 2^{27}$, the scale of Signal's user database, it serves 37K. H_2O_2RAM averages 48K and 38K at those two sizes, but that average is amortized across N requests.

Batched Lookups. At $N = 2^{24}$, across public batch sizes from 1K to 64K, neOMAP serves between 190K and 340K lookups per second. At 64K it reaches 20× Signal's rate for the same batch size and 7.1× H_2O_2RAM's average. Larger batches help because every shard is padded to a fixed capacity that hides how many of the batch's requests actually landed on it. That padding grows sublinearly in the batch size, so the larger the batch, the smaller the share of shard work spent on dummies (Section A.2).

Insertions. Insertions require more work than lookups because each request runs the same public number of Cuckoo relocation rounds, hiding whether it updates a key or triggers a displacement chain (Section 2.2). At $N = 2 \times 10^7$, neOMAP-seq sustains 16K insertions per second, whereas 1K, 8K, and 64K batches reach 66K, 104K, and 132K. The 64K-batch configuration reaches 8.3× the rate of neOMAP-seq (Figure 6(b)). Larger batches spend a smaller share of each shard's fixed capacity on dummy work, and all three outperform H_2O_2RAM

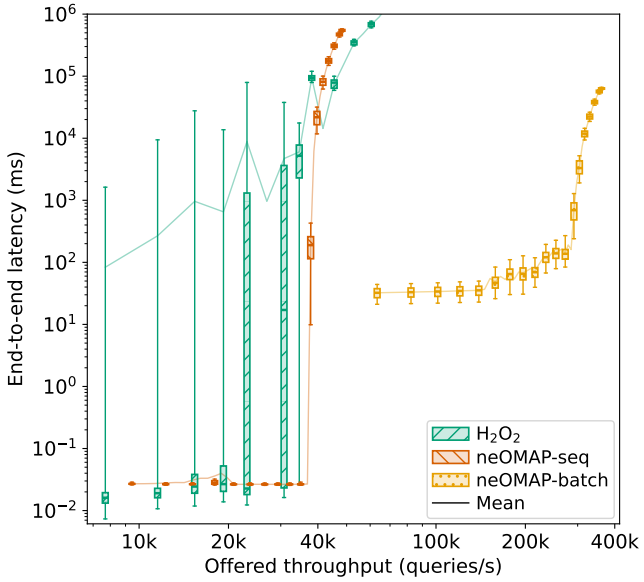


Figure 7: End-to-end latency under increasing offered load (8-byte + 56-byte), for a map of $N = 2^{27}$ entries. A box spans the 25th to the 75th percentile, with the median inside it, and the caps mark the 1st and 99th percentiles.

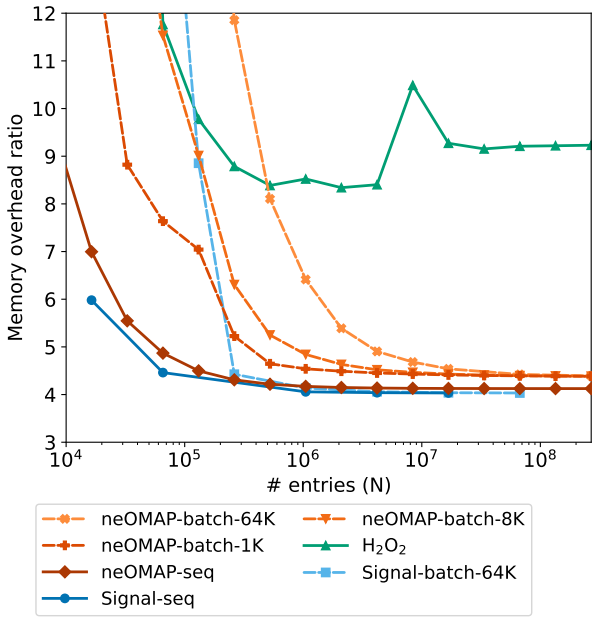


Figure 8: Memory overhead relative to raw data (8-byte + 56-byte). Batched curves count every shard.

at every measured size.

Throughput under page swapping. Every page a map touches beyond its EPC allocation costs an SSD transfer. Signal and H_2O_2RAM have no external-memory layout, so they move more pages per lookup than neOMAP and lose more throughput once the map stops fitting. With a 512 MB EPC the baselines begin swapping near $N = 2^{22}$ (Figure 9(a)). At $N = 2^{24}$, neOMAP-seq sustains 12.5× H_2O_2RAM's lookup throughput, and at $N = 2^{26}$ it sustains 46× Enigmap's. Given a 64 GB EPC, neOMAP-seq reaches 10^9 records, several times what the EPC holds, and still answers 8.9K lookups per second, while neOMAP-batch-64K answers 31K (Figure 9(b)). At that EPC size, once page swapping is incurred, none of Signal, Enigmap, and H_2O_2RAM completes a full run of N requests within 48 hours.

5.3 End-to-End Latency

neOMAP's advantage over H_2O_2RAM is largest under the request pattern of a deployed service, where requests arrive whether or not earlier ones have completed. End-to-end latency has two parts: the work an implementation performs for a request, and the time that request waits behind others. A hierarchical ORAM keeps the first part small on average by deferring work into periodic rebuilds. The periodic rebuilds, however, delay whichever requests arrive while a rebuild is in progress. neOMAP, instead, bounds the work each request performs, so a request waits only for the requests ahead of it. Such difference is not exposed by the saturated-throughput measurements of Section 5.2, but can be important for a service that must respond to requests in real time.

We therefore measure latency as the offered rate rises. A producer thread publishes independent lookup requests on a fixed schedule, a caller thread passes whatever is pending to the implementation's API, and the latency we report spans publication to response. The target rate starts at 25% of each implementation's measured service rate, the rate at which it completes requests. It rises in steps until requests arrive faster than the implementation can serve them.

Figure 7 plots latency against the offered rate. Below 10K queries per second, the sequential configuration averages 0.027 ms against H_2O_2RAM's 83 ms. It holds that average up to 35K, where the baseline reaches 6.0 s. At a higher rate, our batched variant takes over: it adaptively sets the batch size to serve all the requests in the caller queue. At 64K queries per second it averages 32 ms against H_2O_2RAM's 894 s.

H_2O_2RAM's rebuilds account for the huge gap. A hierarchical ORAM periodically merges a prefix of its levels into the next one [57], and the implementation serves no request while a merge runs. At $N = 2^{27}$, its slowest single lookup takes 1,771 s, against an average of 26.4 μs. Its caller is synchronous, so every request published during a merge waits, and the resulting queue outlasts the merge.

Deamortizing the rebuild [32] would bound the work each request performs, at the cost of roughly tripling hierarchy storage, since lookups must be served from the previous version of a level while its replacement is built. Scheduling the two

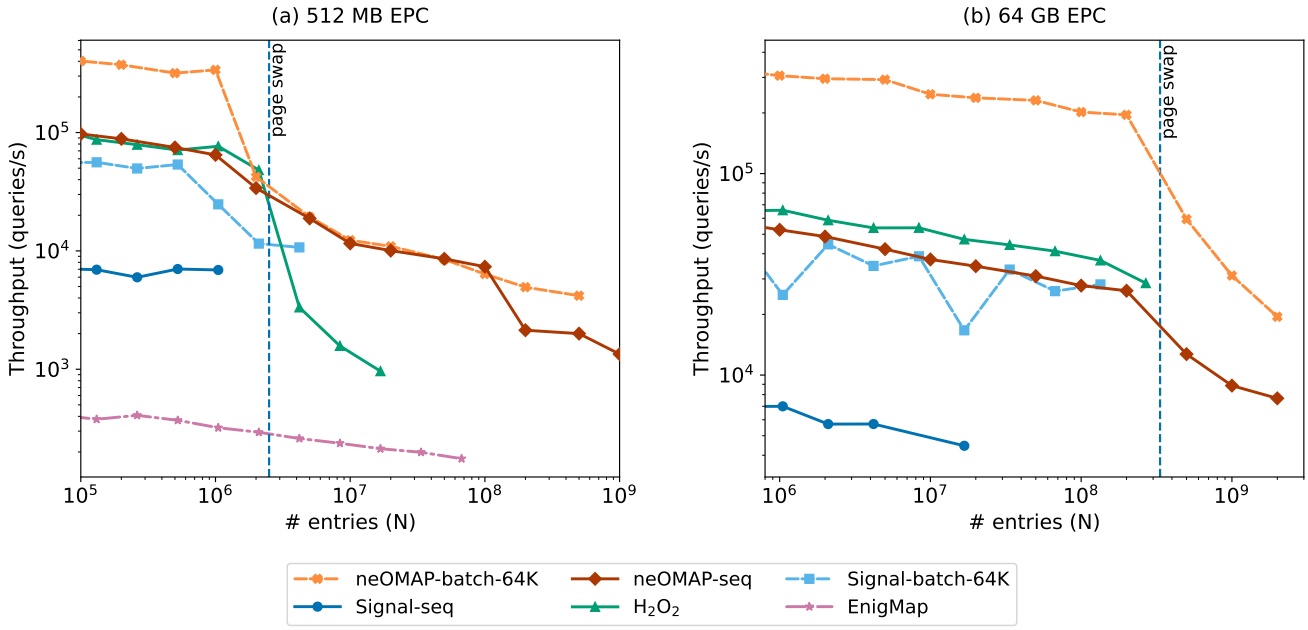


Figure 9: External-memory lookup throughput (8-byte + 56-byte). Each run caps the EPC at the size named above it.

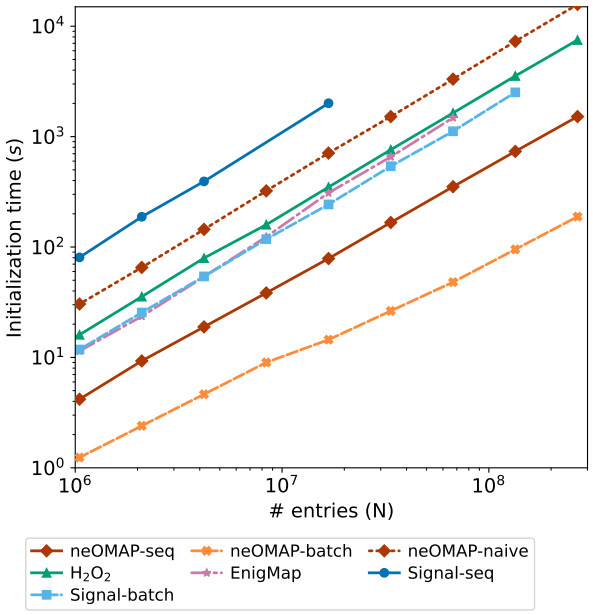


Figure 10: Initialization time (8-byte + 56-byte). neOMAP-naïve inserts records individually.

5.4 Memory Overhead and Initialization Time

Memory overhead. neOMAP matches Signal's footprint and needs less than half of H_2O_2RAM's. At $N = 2^{24}$, neOMAP-seq occupies 4.13× the raw size of the records it holds, against 4.03× for Signal and 9.27× for H_2O_2RAM (Figure 8). Sharding does not change that standing. The three batched neOMAP configurations occupy 4.41–4.54×, next to 4.04× for the 64K-batch Signal configuration.

Initialization time. neOMAP builds a map at Signal's scale in minutes, where H_2O_2RAM and Signal's batched configuration need most of an hour. At $N = 2^{27}$, neOMAP-seq finishes in 12 minutes and neOMAP-batch in 1.6 minutes, against 59 and 42 minutes respectively (Figure 10). The batched build is 26× faster than Signal's. Both neOMAP times come from the bulk initializer of Section 4.2, which settles the Cuckoo placement before building the recursive ORAMs instead of inserting records one at a time.

is also nontrivial, since each request must interleave a fixed slice of rebuild work with its lookup.

References

[1] Meta's oram implementation. https://github.com/facebook/oram.

[2] Mobilecoin's oram implementation. https://github.com/mobilecoinfoundation/mc-oblivious.

[3] Signal private contact discovery implementation. https://github.com/signalapp/ContactDiscoveryService-Icelake.

[4] Technology preview: Private contact discovery for signal. https://signal.org/blog/private-contact-discovery/.

[5] Alok Aggarwal and S. Vitter, Jeffrey. The Input/Output Complexity of Sorting and Related Problems. *Commun. ACM*, 31(9):1116–1127, September 1988. URL: http://doi.acm.org/10.1145/48529.48535, pathdoi:10.1145/48529.48535.

[6] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. Jasmin: High-Assurance and High-Speed Cryptography. In *CCS 2017 - Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1–17, Dallas, United States, October 2017. URL: https://hal.science/hal-01649140.

[7] Yuriy Arbitman, Moni Naor, and Gil Segev. De-amortized cuckoo hashing: Provable worst-case performance and experimental results. In Susanne Albers, Alberto Marchetti-Spaccamela, Yossi Matias, Sotiris Nikoletseas, and Wolfgang Thomas, editors, *Automata, Languages and Programming*, pages 107–118, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

[8] Gilad Asharov, Eliran Eiluz, Ilan Komargodski, and Wei-Kai Lin. Megablocks: Breaking the logarithmic i/o-overhead barrier for oblivious RAM. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, pages 4692–4706. ACM, 2025.

[9] Gilad Asharov, Ilan Komargodski, and Yehuda Michelson. FutORAMa: A concretely efficient hierarchical oblivious RAM. In *ACM CCS*, 2023.

[10] Kenneth E. Batcher. Sorting networks and their applications. In *American Federation of Information Processing Societies: AFIPS Conference Proceedings: 1968 Spring Joint Computer Conference, Atlantic City, NJ, USA, 30 April - 2 May 1968*, pages 307–314, 1968. URL: https://doi.org/10.1145/1468075.1468121.

[11] Elette Boyle, Kai-Min Chung, and Rafael Pass. Oblivious parallel ram. In *Theory of Cryptography Conference (TCC)*, 2015.

[12] Xinle Cao, Weiqi Feng, Jian Liu, Jinjin Zhou, Wenjing Fang, Lei Wang, Quanqing Xu, Chuanhui Yang, and Kui Ren. Towards practical oblivious map. *Proc. VLDB Endow.*, 18(3):688–701, November 2024.

[13] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-abuse attacks against searchable encryption. In *ACM CCS*, page 668–679, 2015.

[14] Anrin Chakraborti, Adam J. Aviv, Seung Geol Choi, Travis Mayberry, Daniel S. Roche, and Radu Sion. ro-ram: Efficient range ORAM with o(log2 N) locality. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society, 2019.

[15] T.-H. Hubert Chan, Yue Guo, Wei-Kai Lin, and Elaine Shi. Cache-oblivious and data-oblivious sorting and applications. In Artur Czumaj, editor, *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2018, New Orleans, LA, USA, January 7-10, 2018*, pages 2201–2220. SIAM, 2018. pathdoi:10.1137/1.9781611975031.143.

[16] T.-H. Hubert Chan and Elaine Shi. Circuit OPRAM: unifying statistically and computationally secure ORAMs and OPRAMs. In *Theory of Cryptography Conference, (TCC)*, 2017.

[17] Graeme Connell. Technology deep dive: Building a faster oram layer for enclaves. https://signal.org/blog/building-faster-oram/.

[18] Natacha Crooks, Matthew Burke, Ethan Cecchetti, Sitar Harel, Rachit Agarwal, and Lorenzo Alvisi. Obladi: Oblivious serializable transactions in the cloud. In *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, pages 727–743. USENIX Association, 2018.

[19] Emma Dauterman, Vivian Fang, Ioannis Demertzis, Natacha Crooks, and Raluca Ada Popa. Snoopy: Surpassing the scalability bottleneck of oblivious storage. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 655–671, New York, NY, USA, 2021. Association for Computing Machinery.

[20] Erik D. Demaine. Cache-oblivious algorithms and data structures. In *Lecture Notes from the EEF Summer School on Massive Data Sets*. BRICS, University of Aarhus, Denmark, June 27–July 1 2002.

[21] John Demme, Robert Martin, Adam Waksman, and Simha Sethumadhavan. Side-channel vulnerability factor: A metric for measuring information leakage. In *International Symposium on Computer Architecture (ISCA)*, 2012.

[22] Martin Dietzfelbinger and Christoph Weidling. Balanced allocation and dictionaries with tightly packed constant size bins. *Theor. Comput. Sci.*, 380(1-2):47–68, 2007. URL: https://doi.org/10.1016/j.tcs.2007.02.054, pathdoi:10.1016/J.TCS.2007.02.054.

[23] Leonid Domnitser, Nael Abu-Ghazaleh, and Dmitry Ponomarev. A predictive model for cache-based side channels in multicore and multithreaded microprocessors. In *Proceedings of the 5th international conference on Mathematical methods, models and architectures for computer network security*, MMM-ACNS'10, pages 70–85, Berlin, Heidelberg, 2010. Springer-Verlag.

[24] Muhammad El-Hindi, Tobias Ziegler, Matthias Heinrich, Adrian Lutsch, Zheguang Zhao, and Carsten Binnig. Benchmarking the second generation of intel sgx hardware. In *Proceedings of the 18th International Workshop on Data Management on New Hardware*, DaMoN '22, New York, NY, USA, 2022. Association for Computing Machinery. pathdoi:10.1145/3533737.3535098.

[25] Saba Eskandarian and Matei Zaharia. Oblidb: Oblivious query processing for secure databases. *Proc. VLDB Endow.*, 13(2):169–183, 2019.

[26] Christopher W. Fletcher, Ling Ren, Albert Kwon, Marten van Dijk, and Srinivas Devadas. Freecursive ORAM: [nearly] free recursion and integrity verification for position-based oblivious RAM. In *ASPLOS*, 2015.

[27] Christopher W. Fletcher, Ling Ren, Albert Kwon, Marten van Dijk, Emil Stefanov, and Srinivas Devadas. RAW Path ORAM: A low-latency, low-area hardware ORAM controller with integrity verification. *IACR Cryptology ePrint Archive*, 2014:431, 2014.

[28] Craig Gentry, Kenny A. Goldman, Shai Halevi, Charanjit Julta, Mariana Raykova, and Daniel Wichs. Optimizing oram and using it efficiently for secure computation. In Emiliano De Cristofaro and Matthew Wright, editors, *Privacy Enhancing Technologies*, pages 1–18, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[29] Craig Gentry, Kenny A. Goldman, Shai Halevi, Charanjit S. Jutla, Mariana Raykova, and Daniel Wichs. Optimizing ORAM and using it efficiently for secure computation. In *PETS*, 2013.

[30] O. Goldreich. Towards a theory of software protection and simulation by oblivious RAMs. In *STOC*, 1987.

[31] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious rams. *J. ACM*, 43(3), May 1996.

[32] Michael T. Goodrich, Michael Mitzenmacher, Olga Ohrimenko, and Roberto Tamassia. Oblivious RAM simulation with efficient worst-case access overhead. In *CCSW*, 2011.

[33] S. Dov Gordon, Jonathan Katz, Vladimir Kolesnikov, Fernando Krell, Tal Malkin, Mariana Raykova, and Yevgeniy Vahlis. Secure two-party computation in sublinear (amortized) time. In *the ACM Conference on Computer and Communications Security, CCS'12, Raleigh, NC, USA, October 16-18, 2012*, pages 513–524. ACM, 2012.

[34] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. Learning to reconstruct: Statistical learning theory and encrypted database attacks. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*, pages 1067–1083. IEEE, 2019.

[35] Tianyao Gu, Yilei Wang, Afonso Tinoco, Bingnan Chen, Ke Yi, and Elaine Shi. Flexway o-sort: Enclave-friendly and optimal oblivious sorting. In *Usenix Security*, 2025.

[36] Yitong Guo, Hongbo Chen, Haobin Hiroki Chen, Yukui Luo, XiaoFeng Wang, and Chenghong Wang. Bolt: Bandwidth-optimized lightning-fast oblivious map powered by secure hbm accelerators. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 2594–2608. Association for Computing Machinery, 2025.

[37] Intel Corporation. Intel SGX for dummies (intel SGX design objectives). https://software.intel.com/en-us/blogs/2013/09/26/protecting-application-secrets-with-intel-sgx.

[38] Adam Kirsch, Michael Mitzenmacher, and Udi Wieder. More robust hashing: Cuckoo hashing with a stash. In Dan Halperin and Kurt Mehlhorn, editors, *Algorithms - ESA 2008, 16th Annual European Symposium, Karlsruhe, Germany, September 15-17, 2008. Proceedings*, volume 5193 of *Lecture Notes in Computer Science*, pages 611–622. Springer, 2008. pathdoi:10.1007/978-3-540-87744-8_51.

[39] Wei-Kai Lin, Elaine Shi, and Tiancheng Xie. Can we overcome the $n \log n$ barrier for oblivious sorting? In *SODA*, 2019.

[40] Chang Liu, Xiao Shaun Wang, Kartik Nayak, Yan Huang, and Elaine Shi. ObliVM: A programming framework for secure computation. In *S & P*, 2015.

[41] Martin Maas, Eric Love, Emil Stefanov, Mohit Tiwari, Elaine Shi, Kriste Asanovic, John Kubiatowicz, and Dawn Song. Phantom: Practical oblivious computation in a secure processor. In *ACM Conference on Computer and Communications Security (CCS)*, 2013.

[42] Andrew Miller, Michael Hicks, Jonathan Katz, and Elaine Shi. Authenticated data structures, generically. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '14, page 411–423, New York, NY, USA, 2014. Association for Computing Machinery.

[43] Brice Minaud and Charalampos Papamanthou. Generalized cuckoo hashing with a stash, revisited. *Inf. Process. Lett.*, 181:106356, 2023. URL: https://doi.org/10.1016/j.ipl.2022.106356, pathdoi:10.1016/J.IPL.2022.106356.

[44] Pratyush Mishra, Rishabh Poddar, Jerry Chen, Alessandro Chiesa, and Raluca Ada Popa. Oblix: An efficient oblivious search index. In *2018 IEEE Symposium on Security and Privacy*, 2018.

[45] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *J. Algorithms*, 51(2):122–144, May 2004.

[46] Enoch Peserico. Deterministic oblivious distribution (and tight compaction) in linear time. arXiv:1807.06719, 2018. URL: https://arxiv.org/abs/1807.06719.

[47] Sofia Ramirez. Signal statistics 2026: Users, finances and encryption adoption. https://sqmagazine.co.uk/signal-statistics/, 2026. Reports 70–100 million monthly active users, per Signal Foundation president Meredith Whittaker (April 2025).

[48] Ling Ren, Christopher W. Fletcher, Albert Kwon, Emil Stefanov, Elaine Shi, Marten van Dijk, and Srinivas Devadas. Constants count: Practical improvements to oblivious RAM. In *USENIX Security*, 2015.

[49] Sajin Sasy, Aaron Johnson, and Ian Goldberg. Fast fully oblivious compaction and shuffling. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, page 2565–2579, New York, NY, USA, 2022. Association for Computing Machinery.

[50] Elaine Shi, T.-H. Hubert Chan, Emil Stefanov, and Mingfei Li. Oblivious RAM with $O((\log N)^3)$ worst-case cost. In *ASIACRYPT*, 2011.

[51] Emil Stefanov and Elaine Shi. Oblivistore: High performance oblivious cloud storage. In *S & P*, 2013.

[52] Emil Stefanov, Marten van Dijk, Elaine Shi, Christopher Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path ORAM – an extremely simple oblivious ram protocol. In *CCS*, 2013.

[53] Afonso Tinoco, Sixiang Gao, and Elaine Shi. Enigmap: External-memory oblivious map for secure enclaves. In *Usenix Security*, 2023.

[54] Xiao Shaun Wang, T-H. Hubert Chan, and Elaine Shi. Circuit ORAM: On Tightness of the Goldreich-Ostrovsky Lower Bound. In *CCS*, 2015.

[55] Xiao Shaun Wang, Kartik Nayak, Chang Liu, T.-H. Hubert Chan, Elaine Shi, Emil Stefanov, and Yan Huang. Oblivious data structures. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014*, pages 215–226. ACM, 2014. pathdoi:10.1145/2660267.2660314.

[56] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*, SP '15, page 640–656, USA, 2015. IEEE Computer Society.

[57] Leqian Zheng, Zheng Zhang, Wentao Dong, Yao Zhang, Ye Wu, and Cong Wang. *H2O2RAM: a high-performance hierarchical doubly oblivious RAM*. USENIX Association, USA, 2025.

[58] Mingxun Zhou, Elaine Shi, and Giulia Fanti. Pacmann: Efficient private approximate nearest neighbor search. In *ICLR*, 2025.

[59] Jinhao Zhu, Liana Patel, Matei Zaharia, and Raluca Ada Popa. Compass: encrypted semantic search with high accuracy. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation*, OSDI '25, USA, 2025. USENIX Association.

Appendix

The appendix is organized into five sections:

A. Algorithmic details. This section gives the algorithms, analyses, and concrete parameters omitted from the main design discussion. It covers deamortized Cuckoo insertion, batched routing and execution, and the external-memory layout.

B. Additional related work and applications. This section places neOMAP in the broader ORAM literature and shows how its map interface supports complete services. It covers systems outside our benchmark set, distributed sharding, and our private Ethereum demos.

C. Additional evaluation results. This section including supporting measurements for the security parameters and performance claims. It covers failure probabilities, the sources of the performance gains, and a variant of insertion without membership privacy.

D. Ethical considerations. This section discusses data sources, stakeholders and potential impact, responsible disclosure and dual-use concerns, and the use of AI tools.

E. Open science. This section lists the artifacts needed to evaluate the paper's contributions and explains how to access their repositories.

A Algorithmic Details

A.1 Deamortized Cuckoo Insertion

Section 2.2 describes how an insertion appends its entry to the pending queue and then performs a fixed number of relocation rounds. This section gives the policy that decides which entry a full bucket evicts, together with the experiment that fixes the queue capacity Q.

The $Q = O(\log N)$ bound cited in Section 2.2 still applies here, but the constant behind it depends on the table's parameters: Arbitman et al. [7] analyze buckets of a single entry, whereas we use buckets of two filled to a load factor of 0.7. We therefore fix Q by measuring the failure probability of our own implementation.

Crowdedness-guided eviction. Because a bucket holds two entries, an insertion into a full bucket must pick which of them to evict. The naive choice is to pick at random. We instead prefer the entry that will be easiest to place again, judging that by how crowded its alternate bucket is. Each table entry e therefore carries a crowdedness score

$$c(e) = 1 + \min_{e' \in B_{\mathrm{alt}(e)}} c(e'),$$

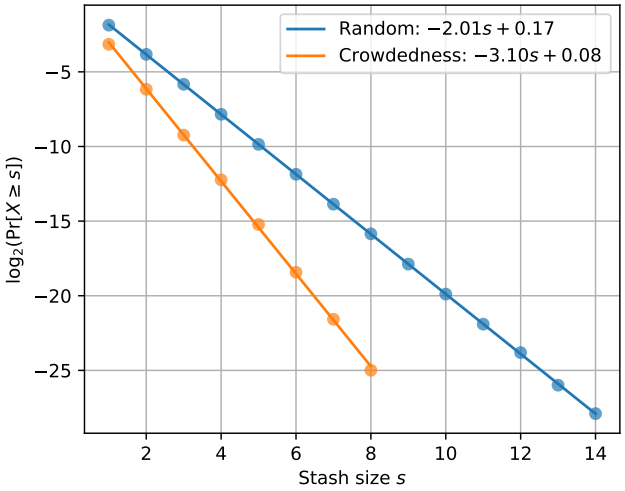


Figure 11: Empirical pending-queue tails for random and crowdedness-guided eviction. The Cuckoo hash table is initialized to store 1 million entries, with a maximum load factor of 0.7 and bucket size 2. The experiment measures the queue-size distribution when inserting the last 50 thousand entries. Markers show measured probabilities and lines show fitted tails. The crowdedness heuristic yields a steeper tail in this experiment.

where $B_{\mathrm{alt}(e)}$ is e's candidate bucket in the other sub-table and an empty slot contributes score zero. A small score thus marks an entry with a relatively uncongested alternate bucket.

When a bucket is full, we evict its least-crowded entry, resolving a tie by a fixed slot order. The score decides only which resident entry the incoming queue head displaces, while the pending queue itself is always served in FIFO order. After an eviction, we refresh the evicted entry's score using the bucket it has just left, which is now its alternate bucket. All other scores are left as they are and may therefore be stale. Recomputing them globally would defeat the point of a constant-work insertion. Even so, the heuristic tends to move entries into the less congested parts of the Cuckoo graph first, which keeps the pending queue shorter.

As shown in Figure 11, the fitted tail slope improves from approximately -2.01 for random eviction to approximately -3.10 with crowdedness. Extrapolating the fitted curves, the queue's budget of $2^{-64}/35$ (Section C.1) requires a pending-queue capacity of $Q = 32$ entries for random eviction and $Q = 21$ entries for crowdedness-guided eviction.

A.2 Details of Batched Parallelization

This subsection fills in the pieces deferred from Section 3: the routing and per-shard insertion algorithms, the padding bound, and the concrete parameters used by the implementation.

A.2.1 Routing a Batch to Shards

In Algorithm 1, step 2 places equal keys adjacent to each other and makes each shard's requests contiguous, allowing step 3 to use a linear scan and step 4 a compaction rather than another sort. Step 7 undoes the routing by replaying the two saved networks backwards, so the answers return to the sorted array without another sort. The router therefore performs two oblivious sorts on m records, a forward and reverse compaction on m records, and a forward and reverse expansion on m_{pad} records. The counting scan examines all n shard counters for every record, so its memory accesses do not reveal the actual counts. The total routing work is

$$O\big(m\log^2 m + m_{\text{pad}}\log m_{\text{pad}} + mn\big).$$

The mn term counts the shard counters that every record examines, and it does not dominate in practice. Sharding is worthwhile only if each shard has a dedicated worker, so n stays well below the $\log^2 m$ factor the sorts already carry. The scan also updates all n counters with vector instructions instead of n separate increments, so counting costs a record only a few instructions.

The capacity c is provisioned so that some c_p exceeds it with probability at most δ. The fixed-range algorithm above and the security analysis below condition on this event not occurring. If it does occur, the router reports a capacity failure and aborts the batch, which is the leakage admitted by Section 1.2.

Stable expansion by reversing OrCompact. OrCompact [49] routes w records, for any w, through a fixed network of $O(w\log w)$ conditional swaps. The length w alone decides which two positions each swap connects and in what order the swaps are issued. Each swap reads one control bit that tells it whether to exchange its two records or leave them in place, and the router obtains all of those bits from the flag vector f marking the live records, by prefix sums over f. No payload enters that computation. The network permutes the array by π_f, which carries the flagged records into the prefix in their original relative order.

A conditional swap is its own inverse, so issuing the same swaps with the same control bits in the opposite order applies π_f^{-1}. The router expands by doing exactly that: it derives the control bits of the length-m_{pad} network from the target vector g, which marks the first c_p slots of every shard range, places the compacted prefix at the front of an array of that length padded with $\perp$, and runs the swaps backwards. The j-th compacted record then lands in the j-th position marked by g. We call this reverse network OrExpand. It is the standard oblivious-distribution primitive [46]. Sasy *et al.* also observe that reversing OrCompact yields expansion [49].

Since the topology and the swap order come from the public length, the addresses touched are the same whatever the flags say, and each swap is executed branchlessly, so the trace does not reveal its control bit either. The router keeps the two flag vectors inside the enclave, f over m records and g over m_{pad}, or $O(m+m_{\text{pad}})$ bits in all. Rederiving the controls and replaying both networks backwards in routing step 7 needs nothing beyond them, and every pass over them is a fixed-length scan.

Shard capacity. Let $u \le m$ be the number of distinct keys after deduplication. Modeling the PRF as a random function, the load c_p of a fixed shard is binomial with mean $\mu = u/n$ and is stochastically dominated by a binomial variable with mean $\mu_0 = m/n$. Let δ be the failure budget assigned to the load balancer and set

$$a = \ln\Big(\frac{n}{\delta}\Big), \qquad c = \left\lceil \mu_0 + \frac{a}{3} + \sqrt{2\mu_0 a + \frac{a^2}{9}} \right\rceil.$$

The Bernstein–Chernoff bound

$$\Pr[c_p \ge \mu + t] \le \exp\left(-\frac{t^2}{2\mu + 2t/3}\right)$$

then gives $\Pr[c_p \ge c] \le e^{-a} = \delta/n$. A union bound over the shards therefore gives $\Pr[\max_p c_p > c] \le \Pr[\max_p c_p \ge c] \le \delta$. The padded size is exactly $m_{\text{pad}} = nc$ and, writing $a = \ln(n/\delta)$, satisfies

$$m_{\text{pad}} = m + O\big(\sqrt{mna} + na\big).$$

For a budget $\delta = 2^{-\kappa}$ and $n \le 2^{\kappa}$ we have $a = O(\kappa)$, so the padding added beyond the m real requests is $O(\sqrt{mn\kappa} + n\kappa)$.

Section C.1 splits a 2^{-64} per-request budget evenly across the 35 components a batched request touches, so we set $\delta = 2^{-64}/35$. A capacity failure is reported for the whole batch, but this bound also applies to any particular request in it.

Note that because repeated keys are removed before counting, an adversary cannot inflate a shard's count by repeating a single key. Further, as long as no capacity failure has occurred, the memory accesses reveal neither the shard assignments nor the counts, so a client colluding with the server cannot use earlier observations to pick keys that are more likely to overflow a shard, since the secret PRF still assigns those keys like a random hash. If a failure is reported, the system picks a new secret PRF and reinitializes the map before continuing.

A.2.2 Executing a Batch Inside a Shard

We spell out batched insertion, the most involved operation. Lookup and erase use the same metadata-table probe and omit or replace the final public update action. For every routed slot, including a dummy request, every operation scans the shard's fixed-capacity pending queue (Section A.1) in full with branchless conditional updates. Thus neither a hit in the queue nor the operation's membership result changes the scan trace.

We first describe why a single sort suffices for all recursion levels. Let $b_i \in \{0,\dots,2^k-1\}$ be the bucket index of request

Algorithm 1 Routing a batch of m requests to n shards.

1: **Tag.** For each request q_i in the batch, let k_i be its key, let v_i be its payload, and let $s_i = h(k_i) \bmod n$ be its shard. Form the record (s_i, k_i, i, v_i), in which i is the position of q_i in the batch.
2: **Sort and deduplicate.** Obliviously sort the records by (s_i, k_i, i) into an array A of size m. In one linear scan, mark the *last* record of each equal-key run as live and mark all other records inactive. Inactive records retain their keys and batch indices so that their answers can be filled and reordered later. The last record of a run has the largest batch index and determines the run's effect under the semantics of Section 3.1.
3: **Count.** In one linear scan over A, accumulate the number c_p of representatives of every shard p. Each step of the scan touches all n counters.
4: **Compact.** Run a stable compaction on A (OrCompact [49]) to gather the live representatives into a prefix A', retaining the flag vector for the reversal in step 7. Preserve the inactive suffix.
5: **Expand.** Build the target flag vector that marks the first c_p slots of shard p's range in an array $\widehat{A}$ of size $m_{\text{pad}} = n \cdot c$. Pad A' with dummy records $\perp$ to that length, stably expand it into $\widehat{A}$ (OrExpand, the reverse of that network), retaining that vector as well. The unused slots hold dummies.
6: **Serve.** In parallel for every shard p, run the batched map procedure of Section 3.4 on the p-th range of $\widehat{A}$, whose length c is public. Real records perform the prescribed accesses, and dummy records visit independent uniformly random paths.
7: **Reverse the expansion and the compaction.** Replay the step-5 network backwards on the answers, combine the resulting representative prefix with the preserved inactive suffix, and replay the step-4 network backwards. This reconstructs an array aligned with A.
8: **Fill duplicates.** In one backward linear scan, copy each representative's answer to the inactive records of its equal-key run.
9: **Restore order.** Obliviously sort by i and return the answers in the caller's order.

i in one metadata table of 2^k buckets, and let L be the number of recursion levels of that table, level L being the table itself. The recursion consumes b from its most significant end, so the logical node visited at level ℓ is $\nu_\ell(b) = \lfloor b/\chi^{L-\ell} \rfloor$ and, for $\ell < L$, the child selected within that node is $d_\ell(b) = \nu_{\ell+1}(b) \bmod \chi$. In particular $\nu_L(b) = b$ is the bucket itself, while the base-case position map inside the enclave stores the identifier of every level-1 node and therefore holds $\lceil 2^k/\chi^{L-1} \rceil$ entries. Because ν_ℓ is monotone in b for every ℓ, sorting by b makes all requests that visit the same node contiguous at every level. We refer to a maximal block sharing $\nu_\ell(b)$ as a level-ℓ *node run*. Runs at level $\ell+1$ subdivide those at level ℓ, so a boundary bit for the former also identifies a distinct-child subrun of the latter. Level L has no position-map child, so there we treat each real request as a singleton *entry subrun* for its key within the fetched Cuckoo bucket.

The following procedure applies to one metadata table. The shard executes the procedure concurrently for its two tables, each in the order induced by that table's bucket indices.

Both sorts lie outside the level loop, and a level itself needs only linear scans and one path access per query. Each metadata table therefore performs two sorts in total, rather than two at every recursion level.

For $\ell < L$, the propagation scan samples one fresh identifier per distinct child. Each entry of $B_{T,\ell}$ stores an updated node together with the node's own fresh identifier, which was sampled while its parent was processed and stored in that parent. At the first recursive level, the parent is the base-case position map. At level L, the old and fresh identifiers instead name the data-ORAM paths from which the record is read and to which it is written. The two tables have separate buffers, so the metadata phase creates one c-slot buffer for every pair (T, ℓ).

Once both tables have restored their outputs to the shard order, a fixed linear pass combines them with the result of the full pending-queue scan. It selects the matching source when the key is present and a dummy old data path when it is absent. Every routed slot then performs one real-or-dummy data-ORAM access. For an absent insertion, the fresh data path is stored with the new metadata entry appended to the pending queue. Every insertion slot then performs the public number of relocation iterations from Section A.1, using dummy entries when necessary. Lookup and erase use the same fixed table, queue, and data-ORAM access pattern, with branchless operation-specific updates. Routing has already made all real keys in the shard distinct, so the data-ORAM phase needs no further deduplication.

Per-tree maintenance. Each metadata tree scans its own buffer $B_{T,\ell}$, and the data ORAM uses an analogous c-slot write-back buffer. A tree X with Π_X paths maintains its own public reverse-lexicographic counter τ_X modulo Π_X [29, 54]. Each buffer slot runs the two-path write-back of Section 3.4 on the adjacent paths τ_X and $\tau_X + 1$: one eviction pass on the first, two on the second. Every slot therefore performs the same three eviction passes and fetches the same two paths, whether its buffer entry holds a real updated node or a dummy.

After a slot, the tree advances its counter by a public stride σ. We require $\sigma \not\equiv 0, 1, -1 \pmod{\Pi_X}$, which makes consecutive two-path windows disjoint so that no slot refetches a path. We also require $\gcd(\sigma, \Pi_X) = 1$, ensuring that the window start visits every residue. Because Π_X is a power of two, the latter condition requires σ to be odd. Over one cycle, every path then appears once as the one-pass path and once as the two-pass path. For the trees to which this external maintenance schedule applies, the implementation uses $\sigma = 3$, which

Algorithm 2 Batched insertion probe for one table inside a shard.

Input: the shard's c queries in the order produced by routing. A dummy query carries a sentinel bucket index that sorts after every real one, so the dummies form a suffix of the sorted order and are never chosen as representatives. A dummy nonetheless performs the same random access as any other non-representative query at every level, so their number stays hidden. The recursion terminates at a small position map that every query scans linearly, so that the lookup leaks nothing through the cache-line trace inside the enclave.
Tag. In one scan, attach to query i its position j_i in the shard range and its bucket index b_i in this table.
Sort by bucket index. Obliviously sort the c queries by b.
for $\ell = 1, \ldots, L$ **do**
    Mark representatives. In one linear scan, mark the first query of each node run as that run's representative. For $\ell < L$, also mark the first query of each distinct-child subrun. For $\ell = L$, mark every real query as the representative of its entry subrun. Dummy queries are never marked.
    Read and remove. Each node-run representative reads and removes its level-ℓ node using the old identifier obtained from its parent. All other queries, dummy or not, read independent uniformly random paths in that Circuit ORAM tree and discard the results.
    Extract identifiers. One forward scan carries the fetched node along its node run by conditional selection. For $\ell < L$, each distinct-child representative extracts its child's old identifier and copies it across the subrun for the next recursion level. For $\ell = L$, each entry representative extracts its key's old data-ORAM identifier, or a dummy identifier if that table does not hold the key.
    Refresh and buffer. The same scan samples one fresh identifier for each distinct child when $\ell < L$, or one fresh data-ORAM identifier per real request when $\ell = L$. It conditionally installs the fresh identifier in the carried node when the child or entry exists and otherwise performs dummy work. The copy carried past the last query of a node run has absorbed all of that run's refreshes, so retain it in this table and level's c-slot maintenance buffer $B_{T,\ell}$. Unclaimed slots hold dummies.
Restore the original order. Obliviously sort the c queries, now carrying the data-ORAM positions obtained from the level-L ORAM, by j.
Output: in the shard's original order, each query holds the old data-ORAM position identifier of its entry, together with the fresh identifier sampled for it, if the entry resides in this table.

satisfies both conditions.

Returning responses before maintenance finishes. A shard returns responses only after it has completed the data-ORAM work for every lookup, or the fixed update work for every insertion or erasure. The enclave first puts the responses back in the caller's order. The caller may use them while the remaining write-backs and evictions run, but the next batch waits until that work is complete. Thus a later batch never sees an unfinished update.

Security of the batch schedule. Conditioned on no capacity failure, the distribution of a batch's trace depends only on m, n, c, the operation type, and the public map configuration. The oblivious networks have public sizes, while the counting, marking, and propagation operations and the pending-queue probes are fixed-length scans. The table probe performs exactly c accesses at each level and at the data ORAM, while an insertion adds the same public number of relocation accesses per routed slot. Maintenance scans every per-tree c-slot buffer under a public deterministic eviction schedule. A node-run representative follows the position identifier obtained from its parent, which was sampled uniformly and independently at the previous access to that block. All other queries at that level, whether they are suppressed duplicate node accesses or dummies, sample uniform paths directly. After within-level deduplication, the adversary thus observes c mutually independent uniform paths per level and shard, with the same distribution for every batch of the same size and operation type.

A.2.3 Additional Parallelization Details

Thread allocation. For sequential requests, neOMAP assigns one thread to each recursion level of each metadata table, plus one thread to the data ORAM (Section 3.2). The resulting count follows from the public configuration. Our implementation uses a recursion branching factor of $\chi = 4$ and stops the recursion at a base-case position map of at most 2^{10} entries that is kept inside the enclave, so a metadata table of 2^k buckets has $L = \lceil (k-10)/2 \rceil + 1$ recursion levels, the last of which is the table itself. A bucket holds two entries and there are two tables, so the two tables provide 2^{k+2} entry slots. Taking the smallest power of two that keeps the Cuckoo load factor below 0.7 gives $k = 26$ at $N = 2^{27}$, hence $L = 9$. With two tables and the data ORAM, the pipeline therefore uses $2L+1$ threads, which is 19 at $N = 2^{27}$.

For batched requests, the cores are instead divided among the n shards, and each shard uses two threads to probe its two metadata tables concurrently. The recursion levels inside a shard are not pipelined, since a shard is allotted too few cores to keep a pipeline full.

A.3 External-Memory Layout

A.3.1 External-Memory Model

When a map is larger than physical memory, its running time depends not only on the computational work it performs, but also on how often it accesses external storage. The classical *external-memory model* captures exactly this distinction [5]. In our deployment, the encrypted pages that do not fit in

memory stay on an SSD and are fetched into a bounded in-memory buffer as needed. We call each such transfer a *page swap*.

A page swap is expensive on two counts. Reaching the SSD is already much slower than reaching memory, and inside a TEE every transfer must also leave the enclave: the enclave makes a system call (an `OCall`) to the operating system, then authenticates and decrypts the page it receives, and it authenticates and encrypts every page it writes back. Avoiding a page swap therefore justifies a substantial amount of extra computation.

Swapping to untrusted DRAM on older enclaves. Older hardware enclaves, such as SGX v1, offer far less protected memory, and a deployment on them may swap the data that does not fit into untrusted DRAM rather than onto an SSD. The model above still applies, and it is still worth transferring at a fixed page granularity, even though DRAM, unlike an SSD, imposes none of its own. Cryptography alone would allow a page as small as a single cipher block, that is, 16 bytes for AES-GCM. Every crossing of the enclave boundary, however, pays a startup cost that is independent of the number of bytes moved: the `OCall` to the OS, and the setup of the authenticated encryption and decryption. Setting the page size at the kilobyte level amortizes that cost over enough useful bytes. Enigmap [53] gives a detailed microbenchmark of these costs.

Notations. Recall from Table 1 that

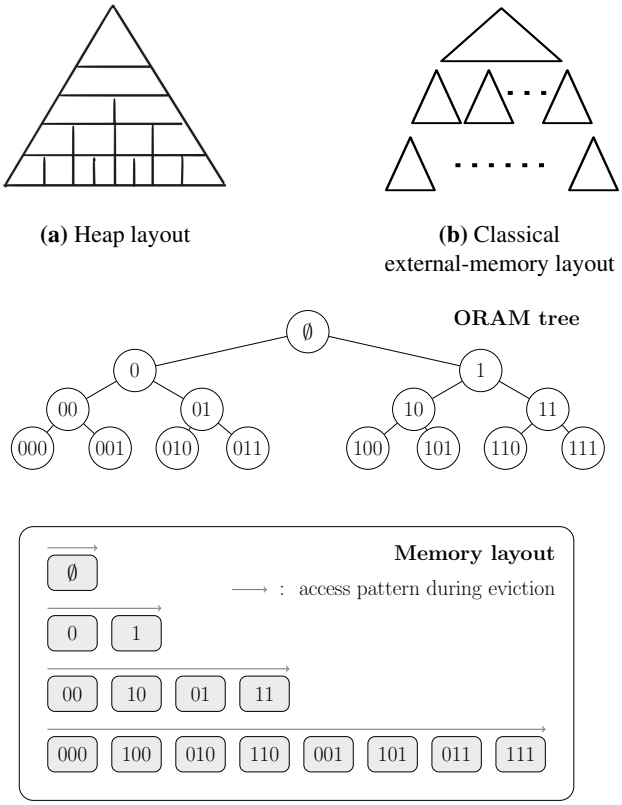
- B is the number of data words in one page;
- M is the number of data words available in the in-memory buffer; and
- N is the database capacity in records, while W is the number of data words in one full data record.

For the layout analysis of one ORAM tree, let n be its number of logical records, let w be the number of words per record, and let b be the number of ORAM-tree nodes that fit in a page. Since every bucket contains a constant number of records, $b = \Theta(B/w)$. After caching the tree top in $\Theta(M)$ words, write

$$H = \Theta\left(\log \frac{nw}{M}\right)$$

for the remaining tree height. A position-map record contains $O(1)$ positions and each position is one word, so $w = \Theta(1)$ for a position-map ORAM, while the data ORAM has $n = N$ and $w = W$.

Oblivious algorithms and external memory. Many earlier oblivious algorithms [40,48,51,54] were written for a cloud or multi-party-computation setting, where the cost that matters is computational work or communication. Their cost models



(a) Heap layout

(b) Classical external-memory layout

(c) Layout of RORAM

Figure 12: Memory layout in prior work. (a) Heap layout (e.g., Oblix [44]): reading and evicting a path both incur $O(H)$ page swaps. (b) Classical external-memory layout [5, 53]: reading and evicting a path both incur $O(H/\log b) = O(\log_b(nw/M))$ page swaps. (c) Layout of RORAM [14]: reading a path incurs $O(H)$ page swaps, and eviction incurs $O(H/b)$ page swaps. The figures are borrowed from prior works [14, 53].

assume that a page holds $O(1)$ words, so they say nothing about the locality of a concrete memory layout, and the cost surfaces in implementations that follow them: Enigmap reports that page swaps account for more than 80% of Oblix's end-to-end running time [44, 53].

More recent systems — Enigmap, RORAM, and Flexway O-Sort — count page swaps alongside computational work [14, 35, 53], following a principle from the classical external-memory literature: once a page has been fetched, do as much work on it as possible before swapping it out. Inside an enclave, where every transfer carries the boundary crossing and the cryptography described above, each avoided swap is worth even more.

A.3.2 Memory Layouts in Prior Work

Prior work uses three memory layouts (Figure 12):

1. **Heap layout** (Figure 12a). Oblix stores the tree in heap order [44], with each trapezoid or triangle in Figure 12a on a separate page. After caching the tree top in $\Theta(M)$ words, reading or evicting a path costs $\Theta(H)$ page swaps.
2. **Classical external-memory layout** (Figure 12b). The classical external-memory layout [5, 20] reads a path in $\Theta(H/\log b) = \Theta(\log_b(nw/M))$ page swaps. Enigmap adopts this layout [53].
3. **Eviction-optimized layout** (Figure 12c). RORAM [14] trades read locality for eviction locality. Circuit ORAM may visit eviction paths in a deterministic reverse-lexicographical (RL) sequence [29], and RORAM stores the nodes at each level in the same order, making the accesses of consecutive evictions contiguous. With $\Theta(M)$ words for the tree top and one page per level, an eviction costs $\Theta(H/b)$ page swaps, while reading a random path costs $\Theta(H)$.

A.3.3 Our Hybrid Layout: Balancing Read and Eviction

Let k denote the number of deterministic RL-order paths that a request touches, besides its read path. A sequential request reads one path, evicts on that path while it is still in the buffer, and puts both of its remaining eviction passes on a single deterministic path, so $k = 1$. A batched request does not evict on its read paths at all. Instead, each of its deferred write-backs uses a window of two adjacent deterministic paths, so $k = 2$ (Section 3.4).

A request thus reads one random path and evicts along k deterministic ones, so the end-to-end latency is lowest when the cost of the two phases is balanced, with the eviction term carrying weight k. Neither prior layout strikes that balance: the classical one is tuned for the read path, RORAM's for the eviction paths. We therefore use a hybrid layout that interpolates between the classical external-memory layout (Figure 12b) and the eviction-optimized layout (Figure 12c). At the page capacities of our implementation, this hybrid scheme reduces the number of page swaps by 1.40× to 1.55× for the batched schedule ($k = 2$) and by 1.18× to 1.28× for the sequential schedule ($k = 1$), compared to the classical external-memory layout (see Table 2). Compared with RORAM's layout, which costs $H + kH/b = H(1 + k/b)$ page swaps per request for one read path and k eviction paths, the reduction ranges from 2.08× to 3.87×.

We collapse subtrees of depth d into super-nodes, giving the collapsed tree fanout 2^d. Within each level, we number the super-nodes by a binary encoding and store them in RORAM's RL order [14]. The original nodes within a super-node are contiguous. Each level of the collapsed tree occupies its own contiguous region, and a page may hold several super-nodes. Unlike the classical layout, ours never requires a super-node to fill a page. As in RORAM, we cache one page per level, so that consecutive evictions, which sweep each region

Table 2: Reduction in page swaps from our hybrid memory layout, relative to the classical external-memory and RORAM layouts, at the four page capacities b arising in our implementation.

k	b (nodes/page)	optimal d	page-count reduction vs. classical	page-count reduction vs. RORAM
1	25	4	1.25×	2.60×
	42	4	1.18×	3.02×
	51	5	1.24×	3.17×
	102	5	1.28×	3.87×
2	25	3	1.44×	2.08×
	42	4	1.40×	2.44×
	51	4	1.51×	2.62×
	102	5	1.55×	3.17×

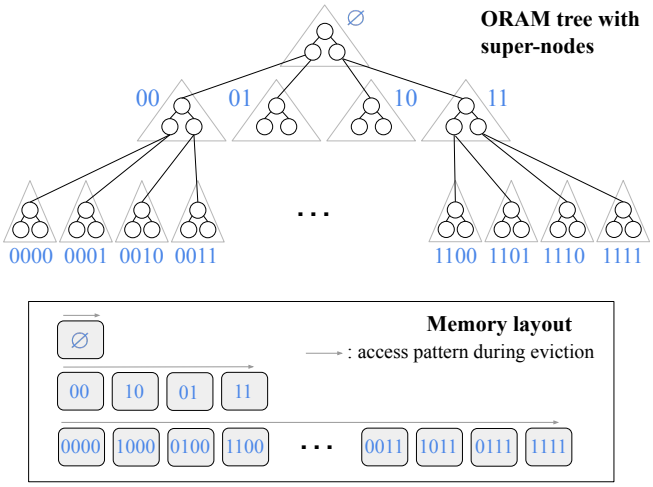


Figure 13: Our hybrid layout. Every subtree of depth 2 is collapsed into a super-node (shown as a triangle).

sequentially, reuse the pages they touch. The layout remains oblivious because the super-node assignment, placement, and eviction order depend only on public parameters and the public RL counter, not on the keys or values stored in the tree. Under this hybrid layout,

- reading a path incurs H/d page swaps, and
- each RL-order eviction incurs $H(2^d - 1)/(db)$ page swaps.

Weighting the eviction term by k and adding the two gives a cost of $\frac{H}{d}\left(1 + \frac{k(2^d-1)}{b}\right)$ per request, which is minimized at the smallest integer d satisfying $(d-1)2^d \geq b/k - 1$, that is, $d \approx \log(b/k) - \log\log(b/k)$. Table 2 evaluates this expression at the four page capacities that arise in our implementation. Here b counts the ORAM-tree nodes that fit in one 4 KB page,

and each node holds two records, so b depends on how large a record is in the tree being laid out:

- For the data ORAM, each record in the ORAM tree stores a hash of the key, the application payload, as well as other metadata needed by our data structure. Concretely, $b = 42$ for Ethereum and $b = 25$ for Signal.
- For the metadata ORAMs, $b = 51$ or 102 depending on the level of recursion and the total memory size N.

The gain is larger for the batched schedule, since its two adjacent deterministic paths per write-back give the evictions more locality to exploit.

In-memory budget under sharding. When the map is sharded, we split the in-memory buffer evenly among the shards, and each shard applies the caching rule of Section 4.1 to its own ORAMs. A shard's reads and evictions are performed by the thread that owns it, and different shards run concurrently. We also reserve enough memory for the largest batch the configuration allows, so that the routing and write-back buffers never spill into external storage. The per-shard page swap counts above therefore still apply.

B Additional Related Work and Applications

B.1 Additional Related Work

Non-baselines. In this paper, we primarily use Signal, Meta, H_2O_2RAM and Enigmap as the baselines for our concrete performance evaluation, and Section 5.1 describes each of them. We do not use every published system as a baseline: some are slower than a baseline we already evaluate, and others optimize a different cost.

- *Batched linear scan*, used in Snoopy [19] and the initial version of Signal [4]. At the dataset and batch sizes we target, a batched linear scan falls far behind even Path ORAM: Enigmap measured it as orders of magnitude slower [53].
- *Earlier enclave-based implementations,* including Mobilecoin's implementation [2], Oblix [44], and ObliDB [25]. We use Signal and Enigmap as the corresponding baselines: Signal's query latency is 10–15× lower than Mobilecoin's, while Enigmap reports a 24–53× speedup over Oblix and ObliDB on Signal/Ethereum-scale datasets.
- *Implementations intended for cloud or cryptographic applications.* Cloud-storage implementations [12, 18, 48, 51] and cryptographic systems such as multi-party computation [33, 40] optimize different costs, so a direct timing comparison would not be meaningful in our setting.
- *Theoretical approaches.* FutORAMa [9] and MegaBlocks [8] evaluate their designs through simulation and analytical estimates rather than full implementations. We instead compare with H_2O_2RAM, an implemented system in the same hierarchical ORAM family [8, 9].
- *FPGA implementation.* The FPGA-accelerated oblivious map of Guo et al. [36] still needs roughly 23–32 ms per query on a database of only one million entries, substantially more than the baselines we do evaluate, such as Signal.

Relation to distributed sharding frameworks. OPRAM and Circuit OPRAM [11, 16], Snoopy [19], and Signal [3] scale horizontally, spreading requests over more servers as the load grows. They solve a different problem from ours: neOMAP makes the map on a single machine faster and puts that machine's cores to work (Section 3). The two are complementary, and a sharded deployment can use neOMAP as the local map at each node, reducing both the number of servers it needs and its total computing cost.

Snoopy instantiates each node with either a batched linear scan or Oblix, and we exclude both for the performance reasons above.

B.2 Additional Applications

Our evaluation treats neOMAP as a plain key–value map. To show that this interface already suffices to build a real service, we also built two open-source demos on top of Ethereum.

ERC20 balance queries. Our first demo lets a client look up an account's balance for an ERC20 token without revealing the account. An updater tracks new Ethereum blocks and keeps the relevant contract state in neOMAP. Before applying an update, the enclave verifies the update's accompanying Merkle proof against the state root committed by the block. Integrity of the imported state therefore comes from Ethereum itself, and neOMAP is responsible only for hiding which entry the client later reads. Source code: https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/oblivious-erc20-state.

Private Ethereum RPC. Our second demo runs Ethereum's `getProof` RPC inside an enclave. A client can ask for account or storage state without revealing the address to the RPC provider, and still receives the usual Merkle proof to verify the answer. Both demos use only the exact-key lookup interface we evaluate in Section 5. Source code: https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/oblivious_node.

Other applications. An oblivious exact-key map is also a natural building block for private credential retrieval. Retrieval-augmented generation needs more than an exact-key map: one would have to layer approximate nearest-neighbor search on top, e.g., using the techniques of Zhu et al. [59] or

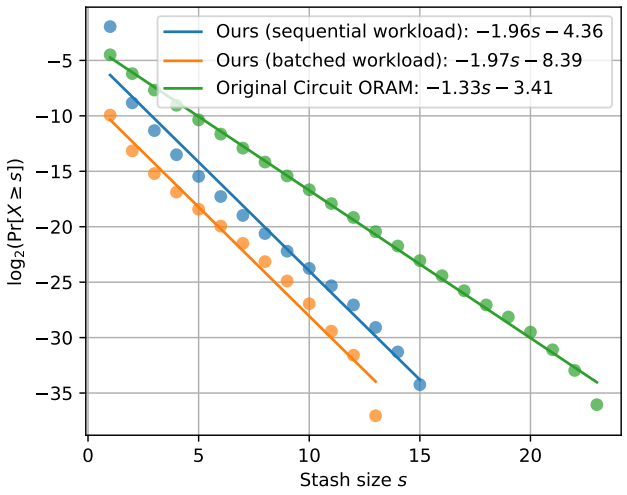


Figure 14: Empirical stash-occupancy tails for three Circuit ORAM eviction schedules ($N = 2^{16}$, $Z = 2$). Markers are measured probabilities; lines are fitted tails, with parameters reported in the legend.

Zhou et al. [58], and the end-to-end leakage of such queries is beyond the scope of this paper.

C Additional Evaluation Results

C.1 Circuit-ORAM Failure Probability

To evaluate neOMAP's third eviction pass (Section 2.1), we ran 32 independent ORAM instances for each schedule. Each instance had capacity $N = 2^{16}$ and bucket size $Z = 2$. On each instance, after a warmup of 10^5 operations, we recorded the stash occupancy for another 4.45×10^9 operations, yielding 1.42×10^{11} samples per schedule.

The fitted tails for both neOMAP schedules decay by roughly $2^{-1.96}$ with each additional stash entry, compared with $2^{-1.33}$ for the original two-eviction schedule. The largest observed stash occupancies were 15 entries for the sequential schedule and 13 for the batched schedule.

Choosing the stash capacity. We divide a 2^{-64} failure budget across the components of one request with a union bound. For a map of up to 2^{32} entries, a batched request touches at most 35 components: 32 recursive position-map ORAMs, the data ORAM, the Cuckoo pending queue, and the load balancer. A sequential request omits the load balancer and touches at most 34 components. With a per-component budget of $2^{-64}/35$, the fitted batched tail first meets the target at 31 entries. The sequential tail first meets its budget of $2^{-64}/34 \approx 2^{-69.09}$ at 33 entries, where its fitted failure probability is $2^{-69.13}$. The implementation uses these empirically derived capacities.

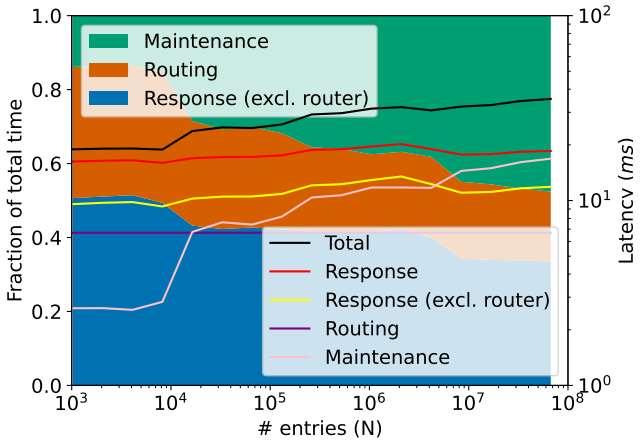


Figure 15: neOMAP latency breakdown ($B = 8192$, 8-byte + 56-byte).

C.2 Performance Breakdown

Section 5 measures whole systems. Here we take one apart and time three pieces on their own: the phases of a batch, the ORAM underneath the map, and the router.

Where a batch spends its time. Increasing the map size adds little to response time; nearly all of the additional batch time falls in the maintenance phase, which runs between batches after responses are ready. Figure 15 splits a batch of 8192 requests over 15 shards three ways: routing, the rest of the response phase, and maintenance. We time routing with the microbenchmark of Table 3 at the same batch size and get the shards' share by subtracting it. From $N = 2^{10}$ to $N = 2^{26}$, a batch grows from 18.9 to 35.4 ms in total, but its response phase only grows from 16.3 to 18.6 ms. Maintenance goes from 14% of the batch to 48% and ends up the largest part. Routing falls from 35% to 19%, because its cost depends on the batch size, not on N.

The underlying ORAM. Strip the map layer away and what is left is a bare ORAM read, which shows the eviction cost of Section 2.1 on its own: Circuit ORAM rearranges an eviction path with a fixed number of linear scans, while the Path ORAM implementations we compare against sort the path. These runs give the enclave enough EPC to hold the whole tree, so nothing swaps to the SSD. Our Circuit ORAM reads 10.4–15.2× faster than the Path ORAM inside Signal for $N \geq 2^{16}$ and uses 1.87× less memory (Figures 16 and 17). It reads 48× faster than Meta's Path ORAM [1] at $N = 2^{24}$, in 2.6× less memory.

Routing a batch to shards. Routing is work that only a batch does, since a sequential request never touches the

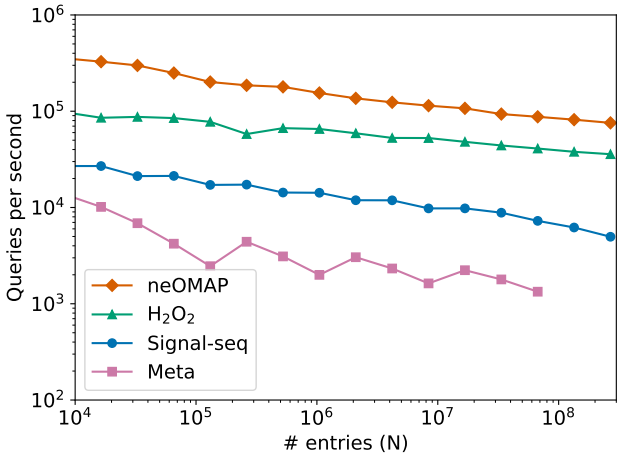


Figure 16: Bare-ORAM read throughput (8-byte + 56-byte).

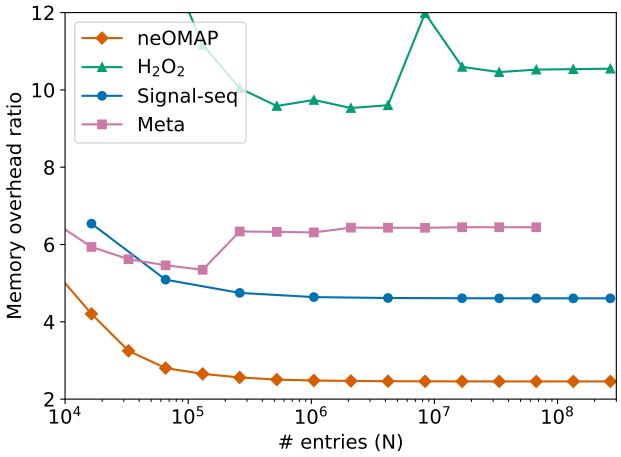


Figure 17: Bare-ORAM memory overhead (8-byte + 56-byte).

router. Table 3 times routing alone, leaving out the per-shard map accesses, which are identical under the neOMAP and Snoopy routing methods. Our router is 1.44–2.11× faster than Snoopy's method across batch sizes from 2^{10} to 2^{20}. The saving is the padding we removed in Section 3.3. Snoopy sorts the batch plus one dummy for every shard slot, roughly twice the number of records sorted by our router. We do not get the full factor of two, because the compaction and expansion passes that take the padding's place are cheaper than a sort but not free.

C.3 Non-privacy-preserving Insertion

Signal's map cannot take insertions while it is serving. Its only insertions are the ones that build the database during initialization, from records the server already has, and those

Table 3: Batch-routing time, neOMAP against Snoopy's routing method [19], for 15 shards with 8-byte keys and 56-byte values.

Batch size	Snoopy (ms)	neOMAP (ms)	Speedup
2^{10}	2.657	1.259	2.11×
2^{12}	7.919	5.097	1.55×
2^{14}	20.60	14.27	1.44×
2^{16}	89.96	53.80	1.67×
2^{18}	375.0	223.7	1.68×
2^{20}	1542	938.9	1.64×

do not hide whether a key was already there. That is why Figure 6(b) leaves Signal out, and why its curve in Figure 18 is that initialization path, timed per entry inserted. To compare on the same footing, we also built neOMAP-public-DB, which likewise does not hide whether an inserted key is already present. Nothing else in the paper uses it.

neOMAP-public-DB inserts the way an ordinary Cuckoo table does. The entry goes into a candidate bucket. If that bucket is full, one of the entries already there moves to its alternate bucket, and the displacements keep going until something lands in a free slot or the stash takes it. There is no deamortization and no padding. An update to a key that is already in the table stops the moment it finds the key, and a scan reads only the part of the stash that is in use. So the time an insertion takes shows how long its chain ran, which is enough to tell an update from a new key.

Lookups still stay oblivious, because the Cuckoo tables and the record store are ORAMs (Figure 1). Dropping the padding changed how many ORAM accesses an insertion makes, but not what one access looks like from the outside: the physical path revealed by an ORAM access is independent of the logical bucket address. A lookup still probes both candidate buckets and scans the stash. The stash scan does show how full the stash is, but that says something about the table, not about the queried key.

An insertion in neOMAP-public-DB costs about half a lookup because most insertions access only one bucket. The oblivious insertion in neOMAP-private-DB costs about 2.2× a lookup, so hiding whether the key was already there costs about 4× (Figure 18).

D Ethical Considerations

Data source and legality. Our measurements depend on the sizes of the keys and values, not on what the records contain. We therefore generate random records with the dimensions of the Signal and Ethereum workloads, and no private dataset is needed for our evaluation.

Signal has expressed interest in eventually incorporating our implementation into its production system, and for that

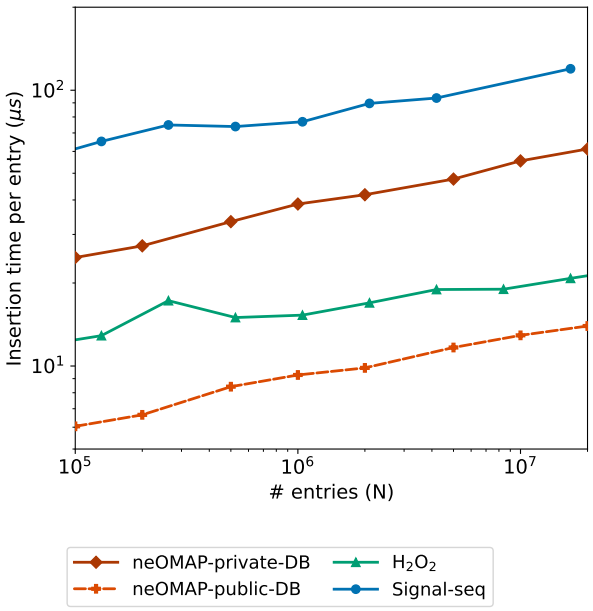


Figure 18: Insertion time per entry with 8-byte keys and 56-byte values. neOMAP-private-DB and neOMAP-public-DB are measured directly; Signal-seq is its total initialization time divided by the number of inserted entries. H_2O_2RAM uses its measured lookup time because its insertion and lookup costs are the same.

purpose gave us a mock database of garbage records whose dimensions match its current user database. We evaluated neOMAP on this mock database and reported the results to Signal.

Stakeholders and potential impact. The immediate stakeholders are the organizations that provide access to large sensitive datasets (e.g., Signal, Ethereum, Meta), the individuals whose records those datasets hold, and the clients whose queries may reveal sensitive interests. By hiding the memory accesses of each map operation, neOMAP protects both the stored records and the clients' queries from the host, and it lets a service provider compute over a large sensitive dataset without learning anything about an individual user. It does not decide which records an operator may collect or who may query them, and those policies remain the operator's responsibility.

We believe that our work will lead to real-world impact in the near term: we have been collaborating with Signal and the Ethereum Foundation, both of which have *expressed interest in deploying neOMAP.*

Responsible disclosure and dual-use concerns. While working on this paper, we identified several flaws in Signal's production implementation that may leak information. We reported them privately to Signal's technical lead. Signal had already discovered a subset of them independently, and has since published a new version that uses formal verification tools to systematically defend against such flaws. Since Signal has fixed the issues in their production system and open-source code, we do not believe that our work introduces new dual-use concerns. The fix was not free, though: our experiments show that their new implementation has 50% lower throughput than the old one. Our work can help improve the performance of their system without compromising security.

Use of AI tools. We designed the algorithms, built the implementations, ran the experiments, and wrote the manuscript ourselves. AI tools helped us polish the language and choose the system name neOMAP. We also used them to convert figures into TikZ: some were first drawn by hand by the authors, and others were adapted from prior work to illustrate prior approaches, in which case the caption cites and credits the source.

E Open Science

Artifacts are available at the following URLs (copied to sanitized URLs for anonymous submission):

- **neOMAP C++ version:** https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/neomap-cpp
- **neOMAP Rust version:** https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/neomaprust
- **ERC20 balance checker demo:** https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/oblivious-erc20-state
- **Ethereum wallet server:** https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/oblivious_node
- **Benchmarking repository for comparing various schemes:** https://github.com/neomap-anonymized/neOMAP-anonymized/tree/main/benchmarks